



Technical Introduction to PSOC™ Edge E84 features

Customer training workshop (CTW)

January 2026



Prerequisites

- Training prerequisites
 - For an introduction to PSOC™ MCUs, including a getting started guide to ModusToolbox™, visit <https://www.infineon.com/product-information/psocdeveloper>
 - For PSOC™ Edge trainings, from beginner to advanced, visit [PSOC™ Edge E84 Training Collection](#)
- Development tools for labs (see training manual for links)
 - ModusToolbox™ v3.6
 - Eclipse IDE for ModusToolbox™ v2025.4.0
 - Edge Protect Security Suite v1.6
 - ModusToolbox™ Programming Tools v1.5.0
 - KIT_PSE84_EVAL_EPC2 BSP v1.0.0
 - Terminal Emulator
 - KIT_PSE84_EVAL

Prerequisites (contd.)

Additional development tools

- Audio labs
 - Install [LLVM for Arm®](#) v19.1.5
 - Set the environment variable CY_COMPILER_LLVM_ARM_DIR to point to the installation directory
 - Create an account for [DEEPCRAFT™ Voice Assistant](#)
- Graphics labs
 - Install [EEZ-Studio](#) v0.24.0 or later
 - Install [Python 3](#)
 - Install "pypng" and "lz4" Python modules
 - `python -m pip install pypng==0.20220715.0`
 - `python -m pip install lz4`
 - Install [Pngquant](#) v2.17.0 or later
 - Download binaries, unzip, and add folder to path environment variable
- Machine learning labs:
 - Install [DEEPCRAFT™ Studio](#) v5.6.3681 or later
 - Create an Imagimob account

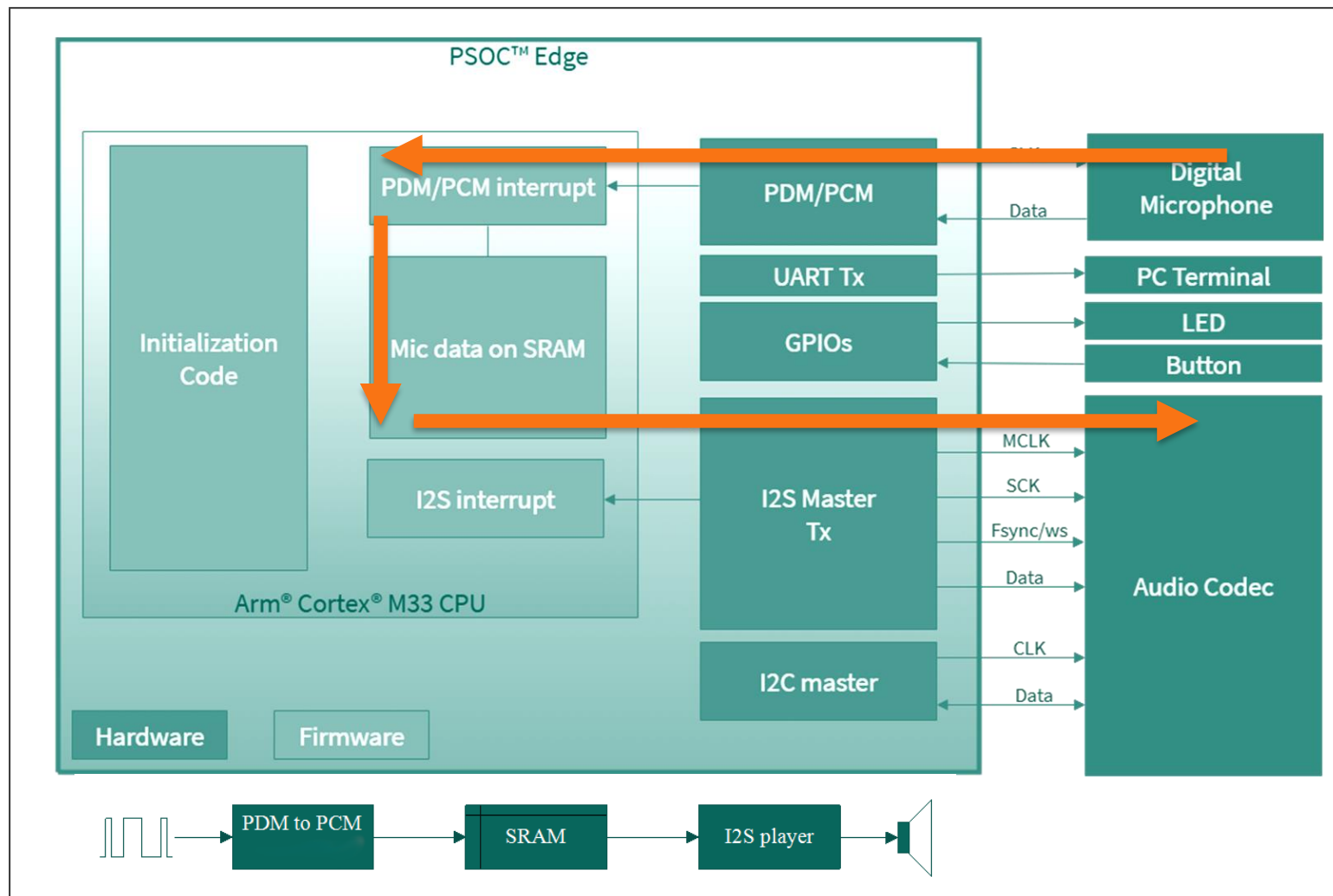
Table of contents

1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

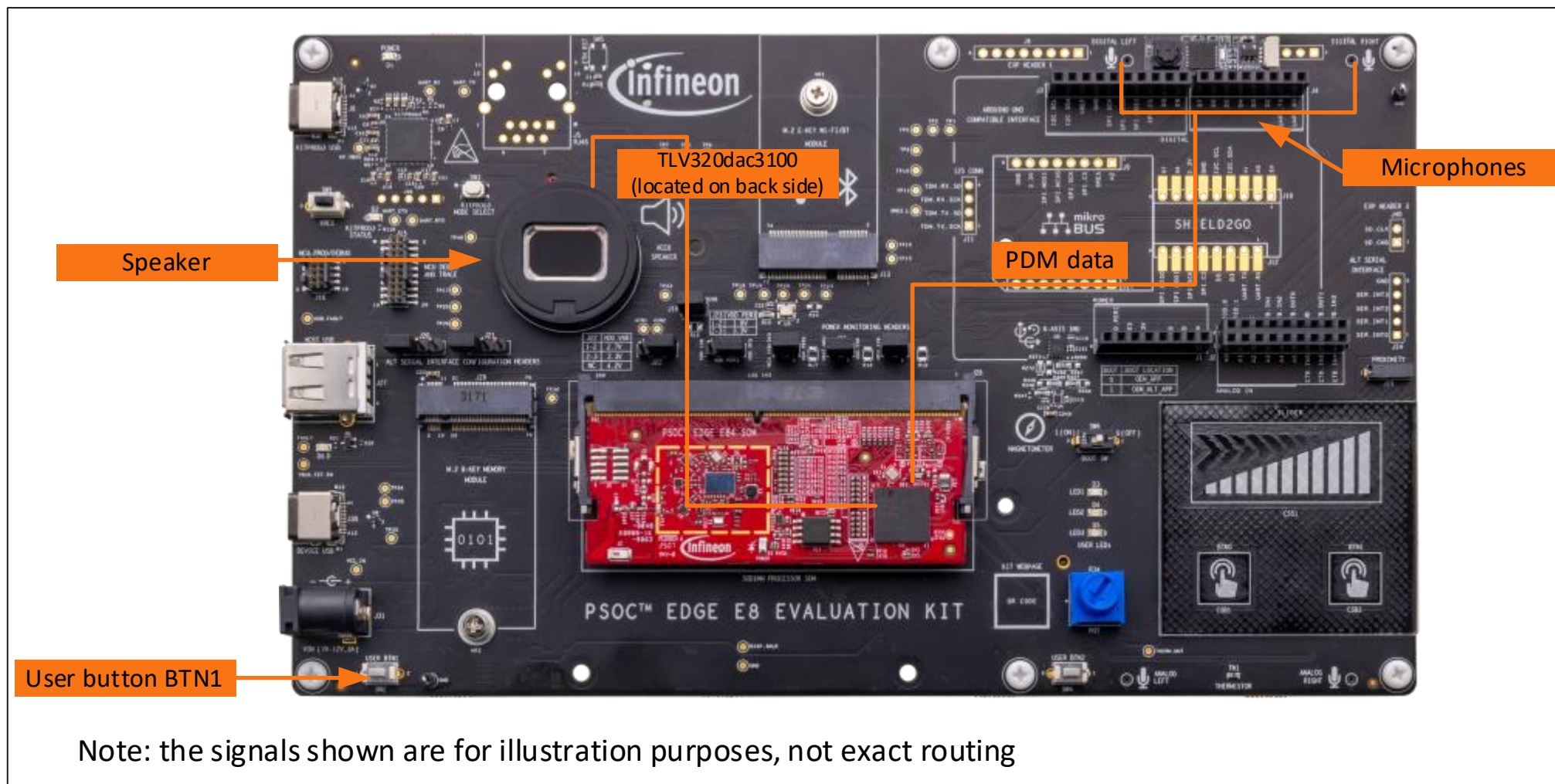
Table of contents

1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

PDM to I2S overview



Hardware overview



Lab 1: PDM to I2S example

What will you learn?

- How to route pulse-density modulation (PDM) audio data to the inter IC sound (I2S) interface
- How to record a short audio sample from a microphone, and then play it on a speaker or headphone
- How to modify settings, such as volume

Observe output on the PSoC™ Edge E84 MCU dev kit

1. Press and hold user button 1 to begin recording.
2. Release and listen to recorded audio.



The screenshot shows a Tera Term VT window titled "COM41 - Tera Term VT". The window contains the following text:

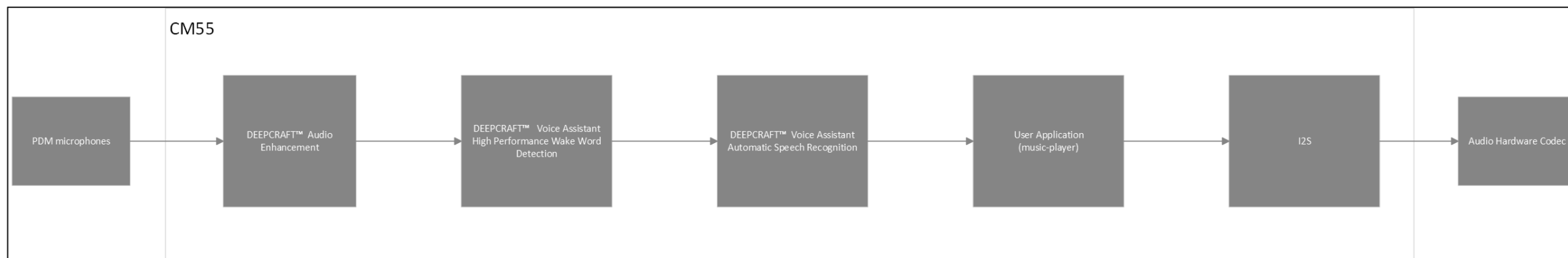
```

***** PSoC Edge MCU: PDM to I2S Code Example *****
Hold User button 1 to start recording.
Release to stop recording.
Listen to the recorded audio.

```

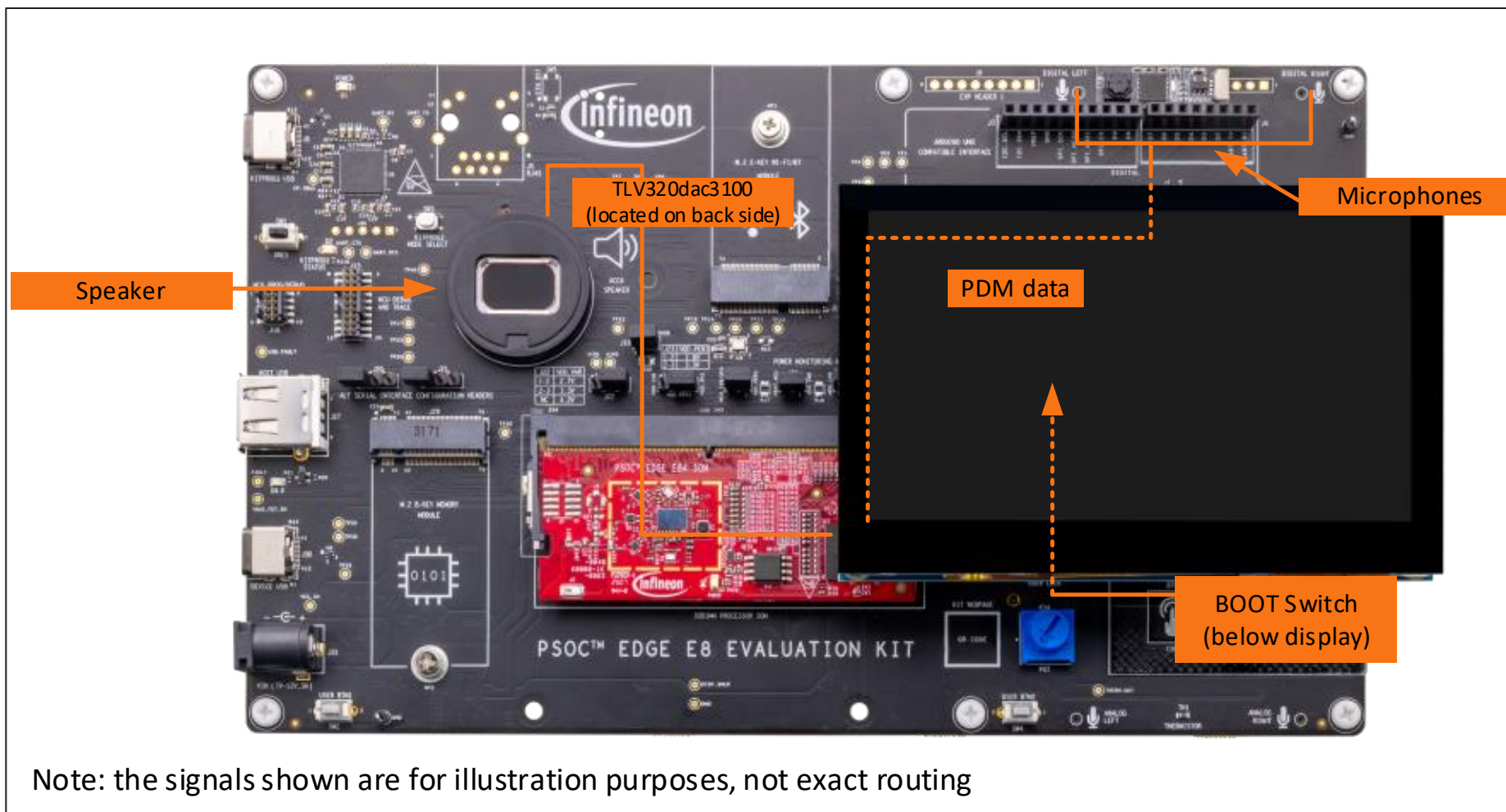
The text is displayed in a monospaced font on a black background. The window has a standard menu bar with "File", "Edit", "Setup", "Control", "Window", and "Help".

Mains-powered local voice overview

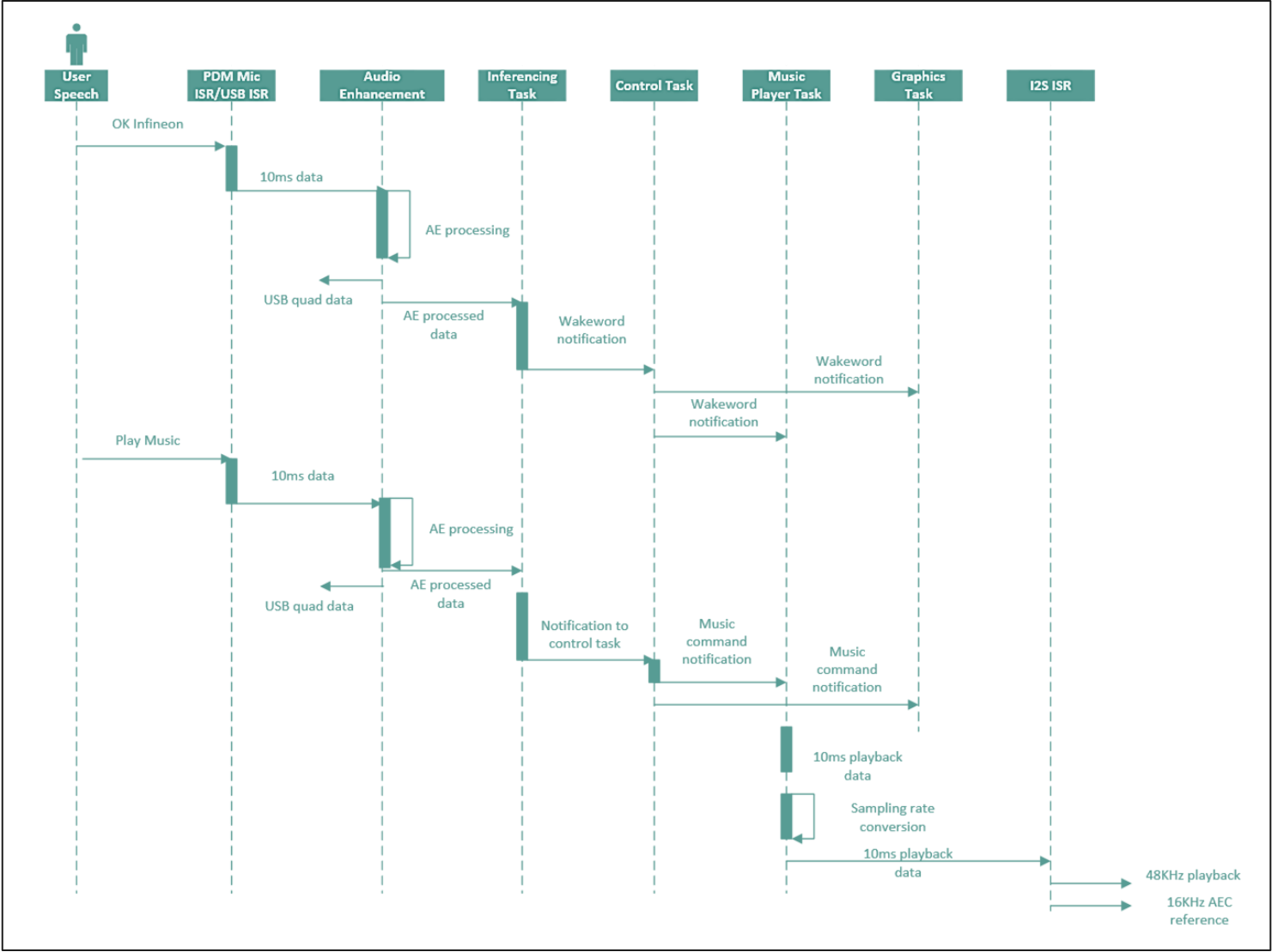


Audio pipeline

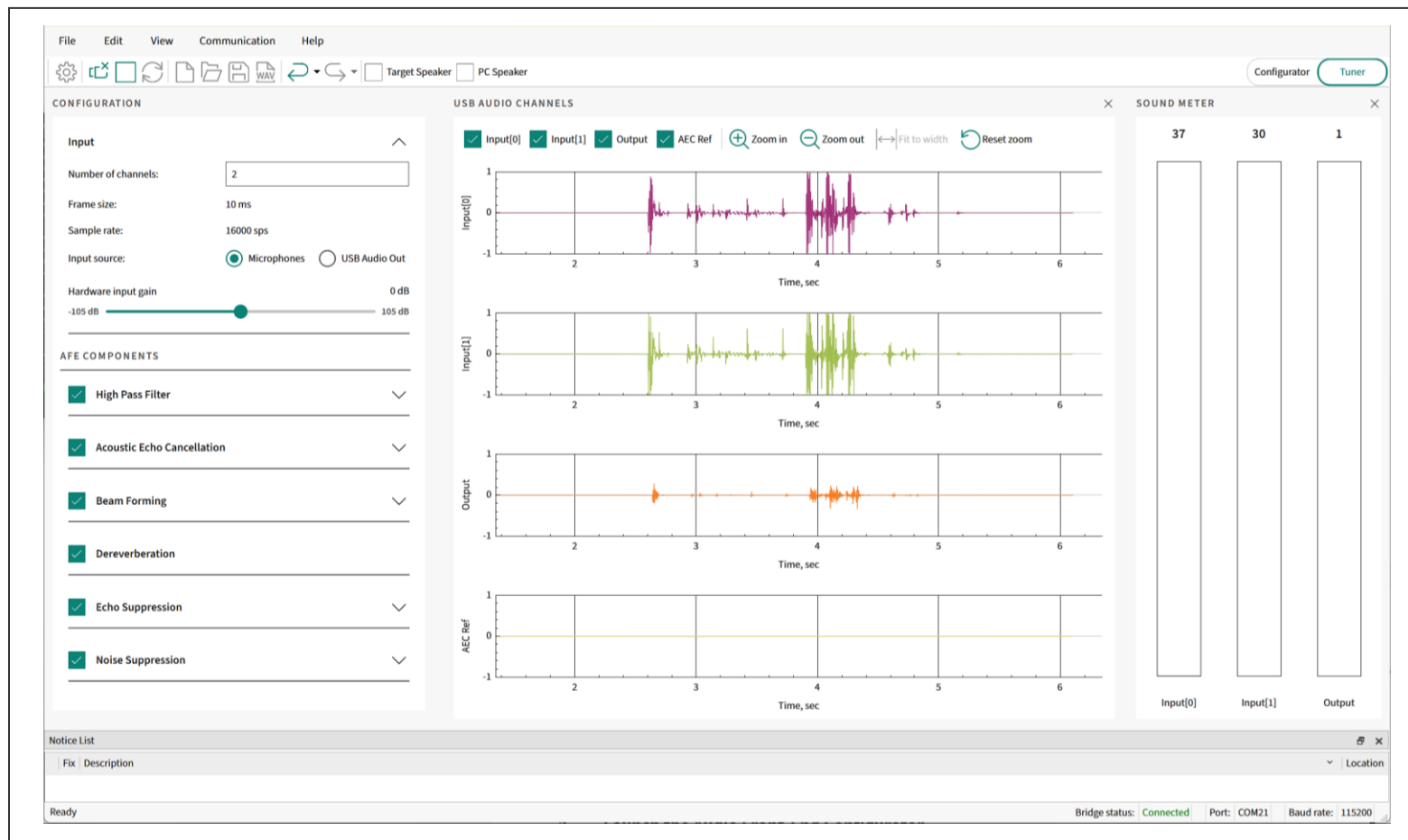
Hardware overview



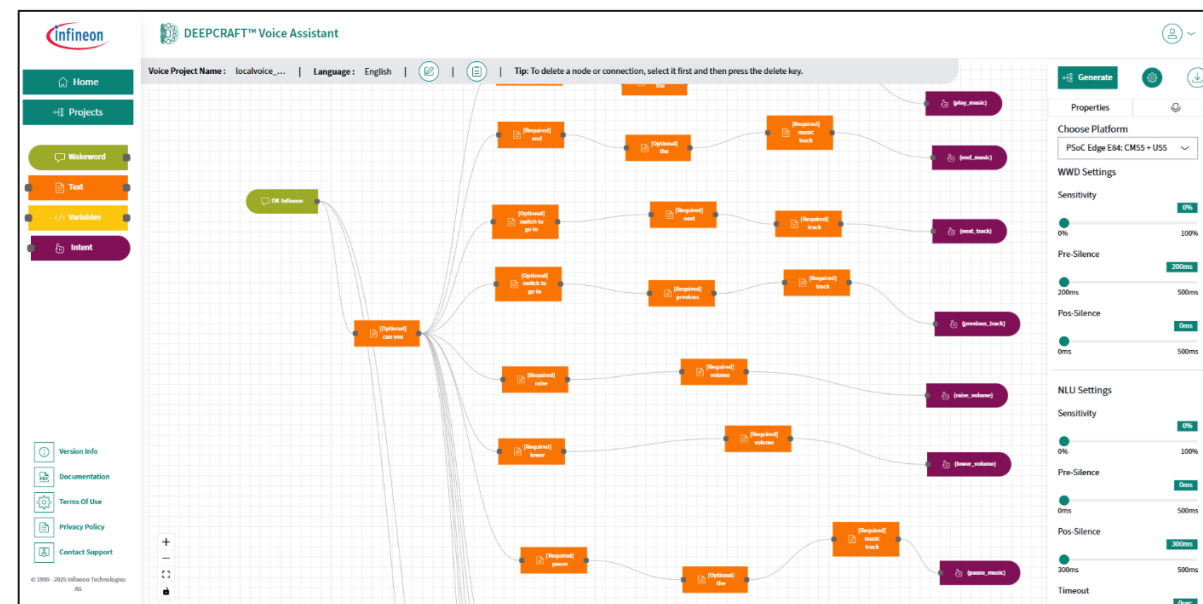
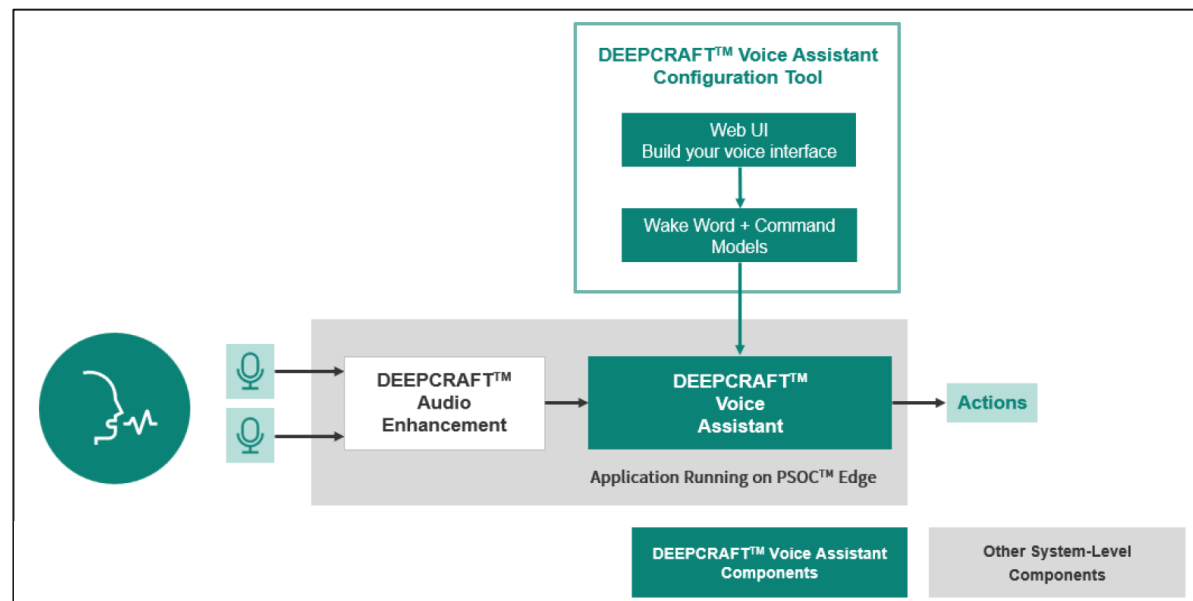
Code flow



ModusToolbox™ Audio Front-End Configurator



DEEPCRAFT™ Voice Assistant



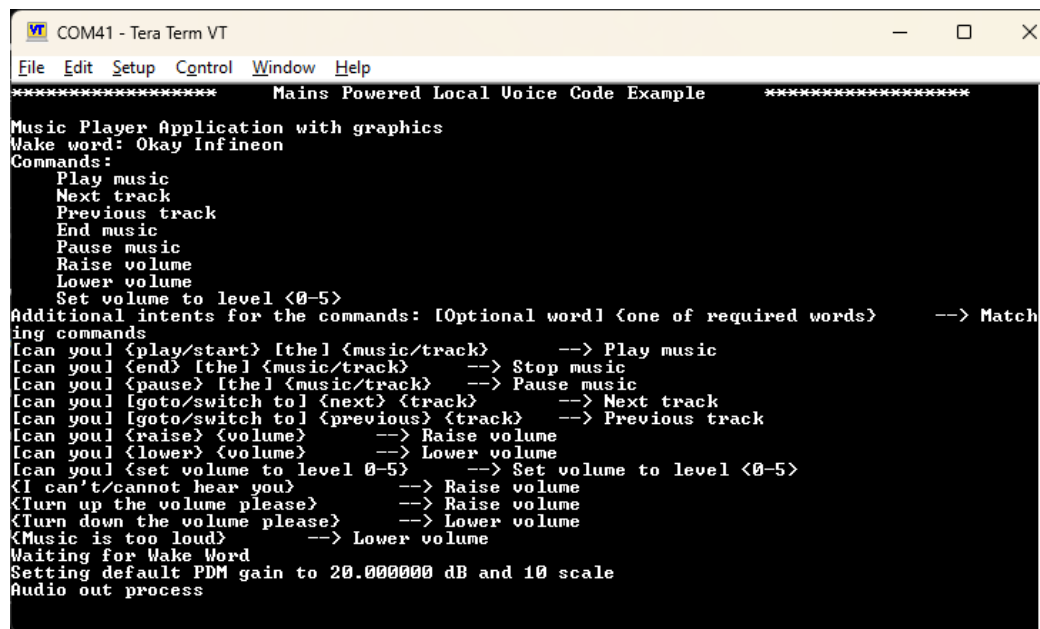
Lab 2: Mains-powered local voice

What will you learn?

- How to process audio using the audio front-end (AFE) components
- How to detect voice commands and activate wake-word detection using Infineon libraries and DEEPCRAFT™ Voice Assistant
- How to use DEEPCRAFT™ Voice Assistant to modify the wake-word and commands

Observe output on the PSoC™ Edge E84 MCU Dev Kit

1. Speak the wake-word “OK Infineon”, followed by an ASR command, such as “Play music”.
2. Observe the output on the terminal and try different intents.



```

COM41 - Tera Term VT
File Edit Setup Control Window Help
***** Mains Powered Local Voice Code Example *****
Music Player Application with graphics
Wake word: Okay Infineon
Commands:
  Play music
  Next track
  Previous track
  End music
  Pause music
  Raise volume
  Lower volume
  Set volume to level <0-5>
Additional intents for the commands: [Optional word] <one of required words> --> Match
ing commands
[can you] <play/start> [the] <music/track> --> Play music
[can you] <end> [the] <music/track> --> Stop music
[can you] <pause> [the] <music/track> --> Pause music
[can you] <goto/switch to> <next> <track> --> Next track
[can you] <goto/switch to> <previous> <track> --> Previous track
[can you] <raise> <volume> --> Raise volume
[can you] <lower> <volume> --> Lower volume
[can you] <set volume to level 0-5> --> Set volume to level <0-5>
<I can't/cannot hear you> --> Raise volume
<Turn up the volume please> --> Raise volume
<Turn down the volume please> --> Lower volume
<Music is too loud> --> Lower volume
Waiting for Wake Word
Setting default PDM gain to 20.000000 dB and 10 scale
Audio out process
  
```

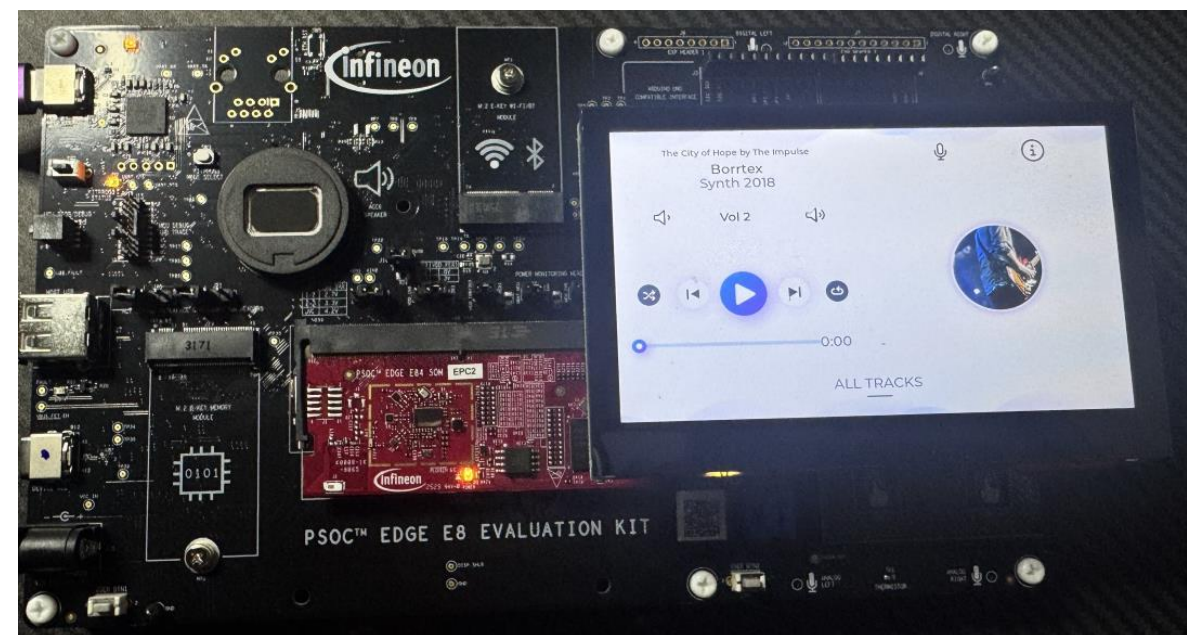
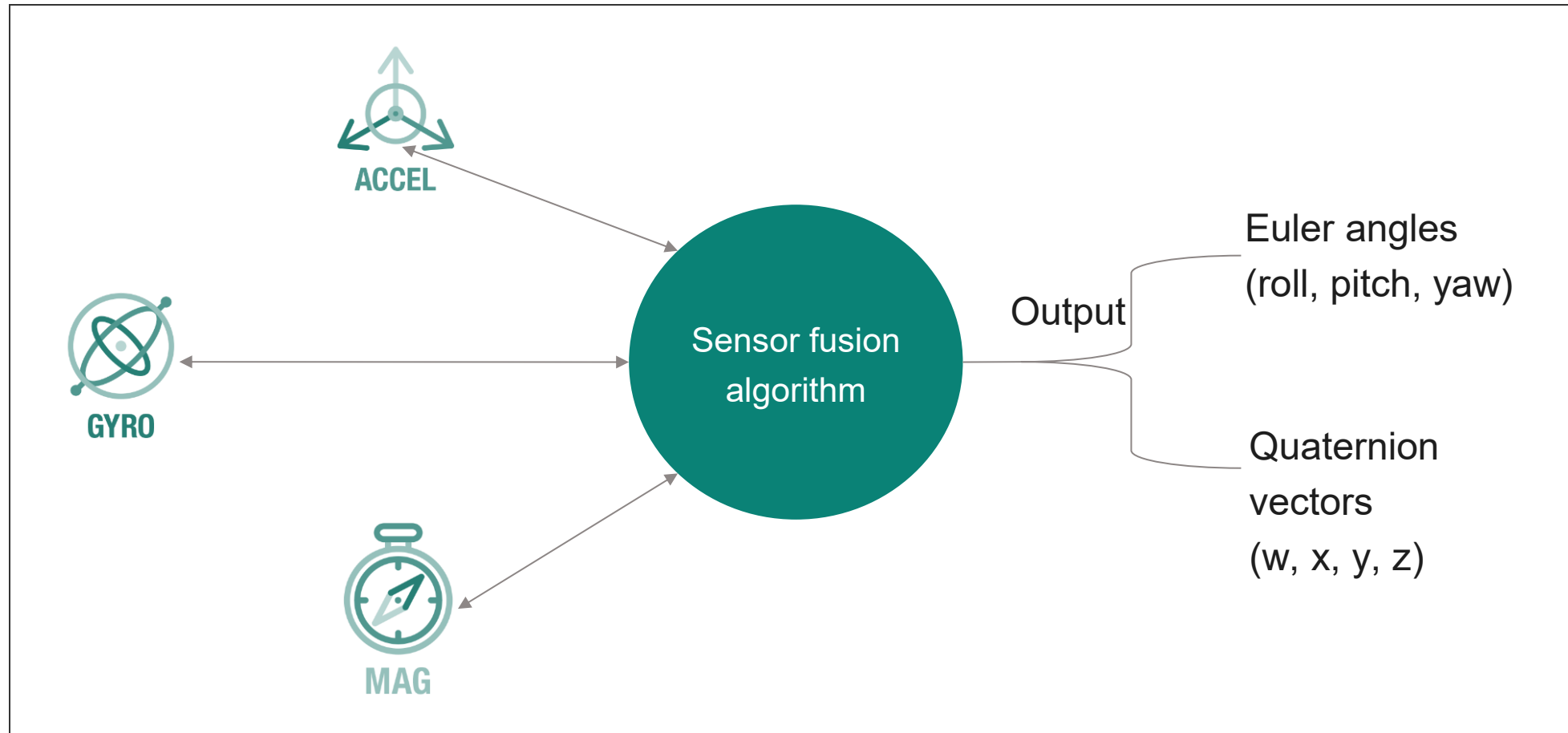


Table of contents

1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

Sensor fusion



Typical applications



Mobile devices



Smart watches



Security cameras



PC peripherals



Robotics



Gaming controllers

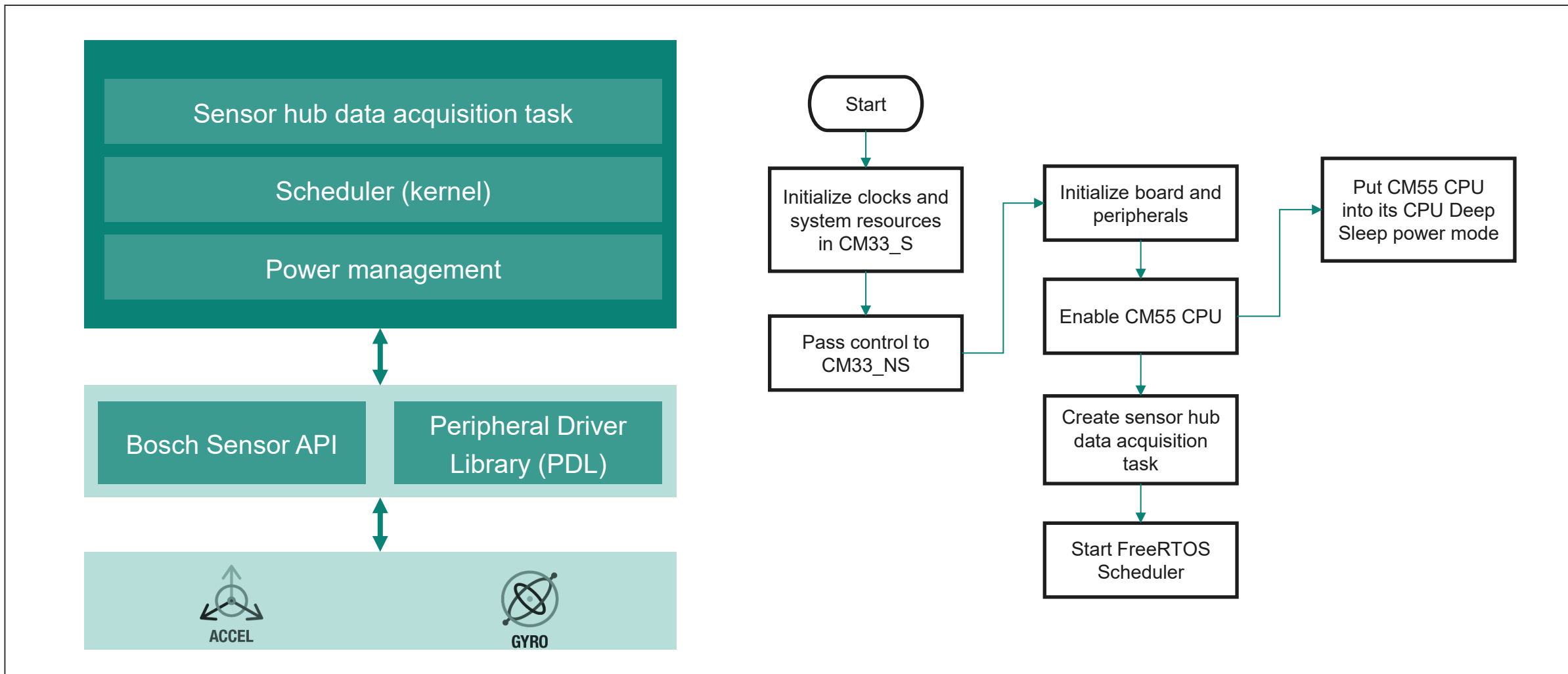


Smart homes

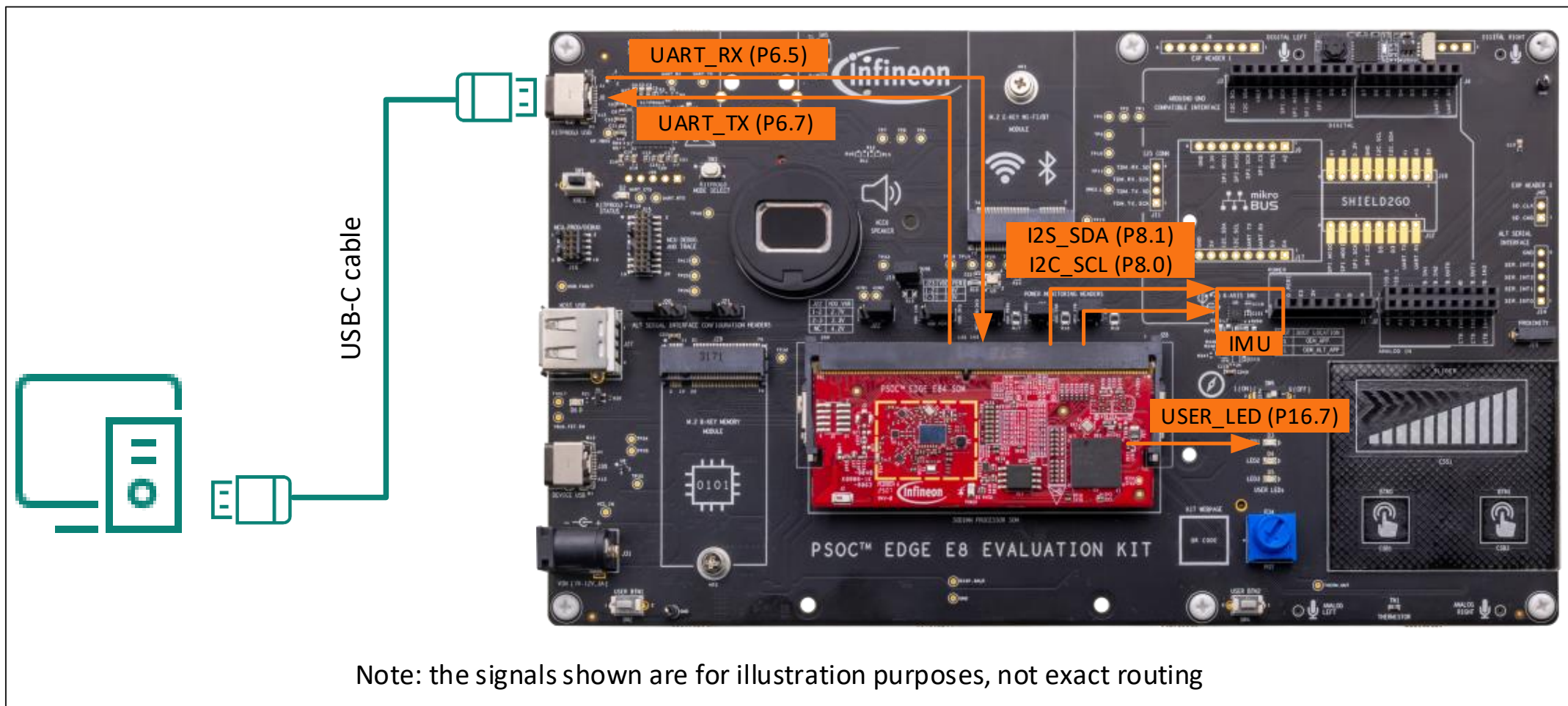


Smart appliances

Sensor hub IMU



Hardware overview



Lab 3: Sensor hub IMU

What will you learn?

- How to collect and process data from various sensors in real time
- How to acquire accelerometer and gyroscope data from BMI270
- How to obtain temperature data
- How to enable sensor fusion to calculate Euler angles and quaternion vectors

Observe output on UART terminal

```

PSOC Edge MCU: Sensor hub IMU
*****
Code example configured in data acquisition mode █
Accelerometer:
x: 0.008978, y:-0.117914, z: 9.813833

Gyroscope:
x: 0.004260, y:-0.007455, z: 0.004260

Orientation = ORIENTATION_UP
█

```

1. Default output

```

*****
PSOC Edge MCU: Sensor hub IMU
*****
Code example configured in data acquisition mode
Accelerometer:
x: 0.058059, y:-0.161609, z: 9.810840

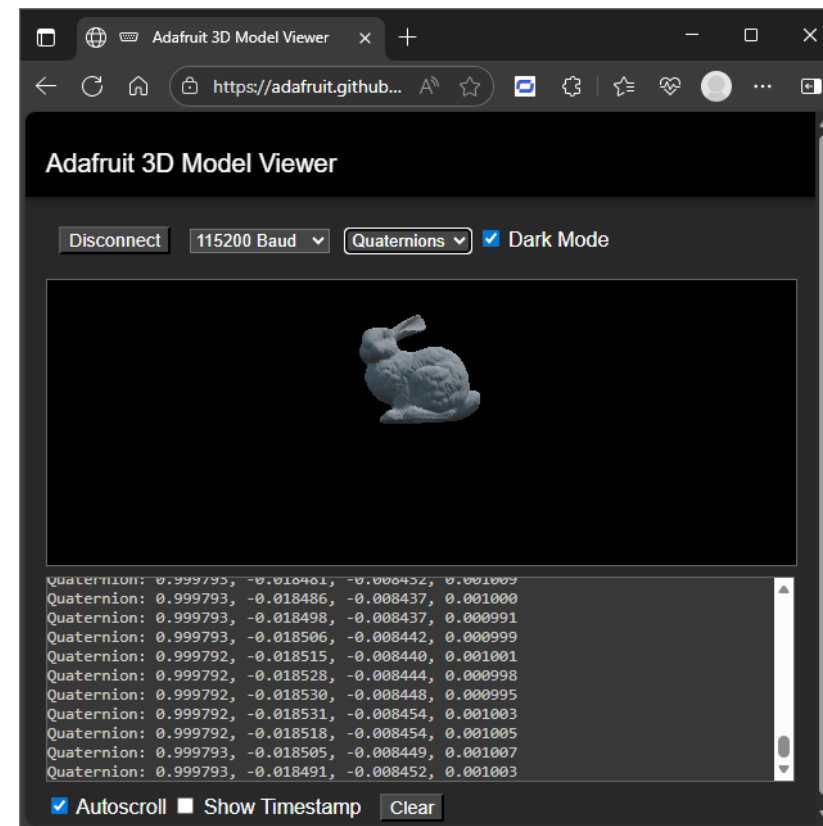
Gyroscope:
x: 0.001065, y:-0.006390, z: 0.003195

Temperature = 25.00 Degree Celsius

Orientation = ORIENTATION_UP
█

```

2. Adding temperature measurements



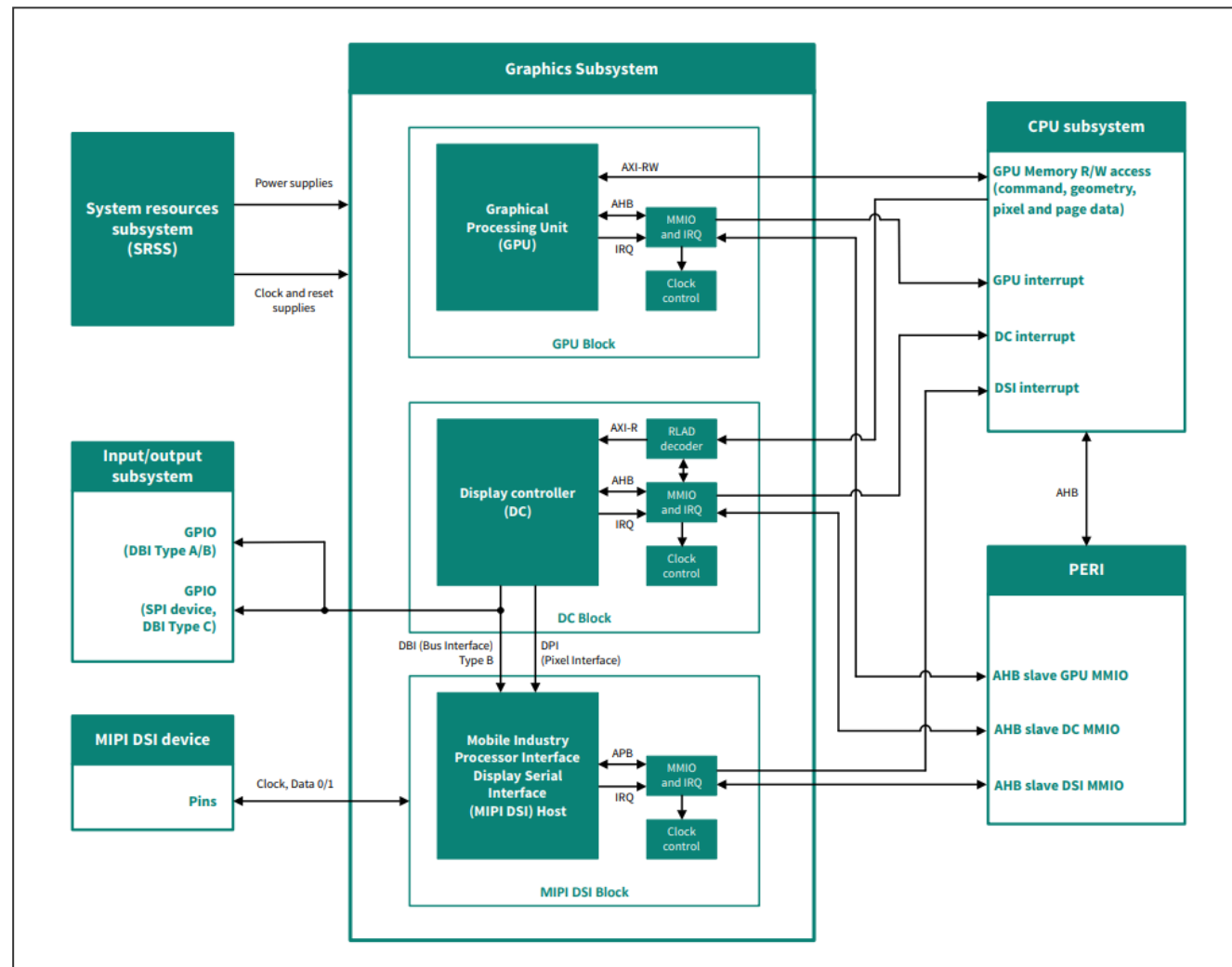
3. Observing quaternion vectors using sensor fusion library

Table of contents

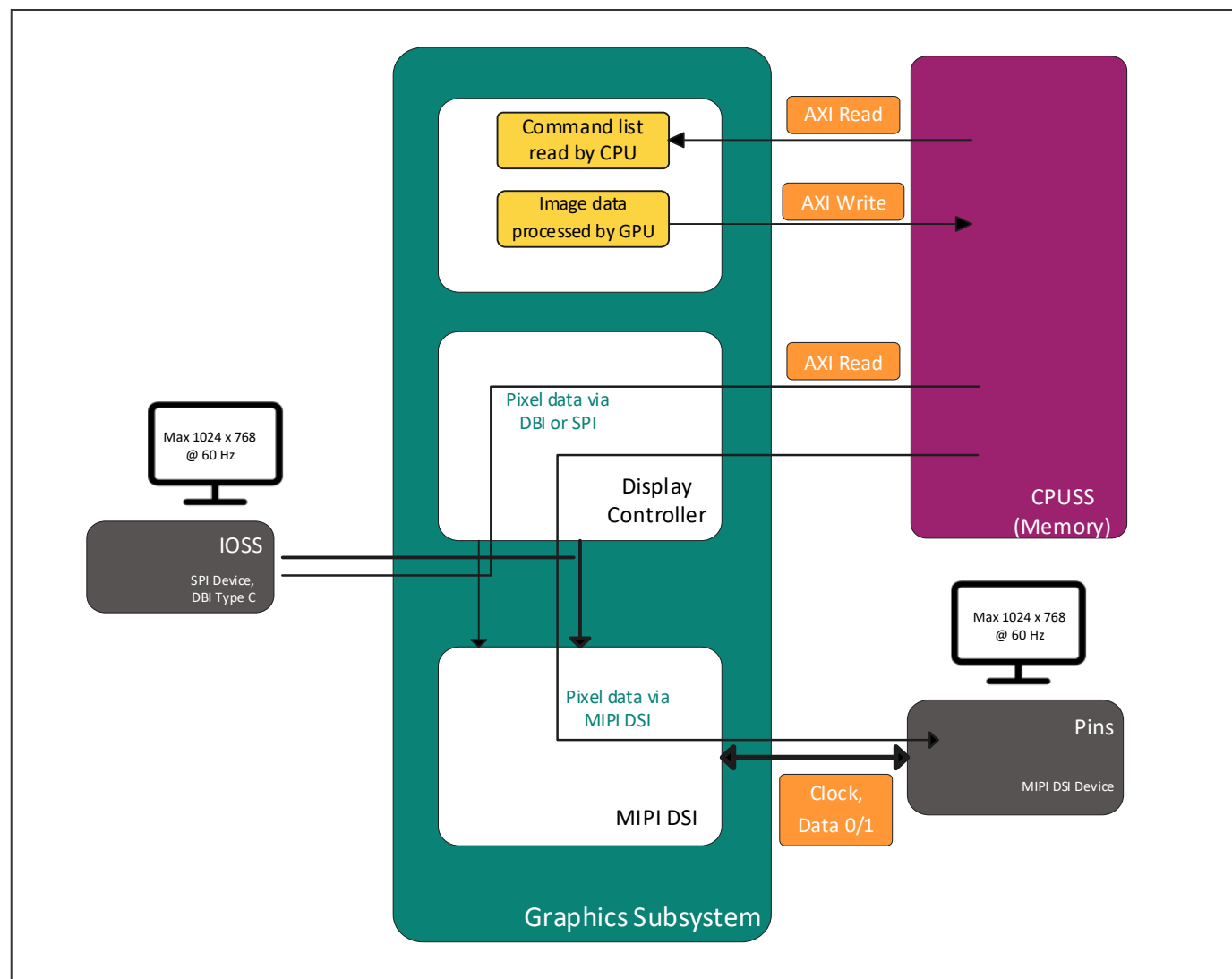
1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

Graphics subsystem (GFXSS) overview

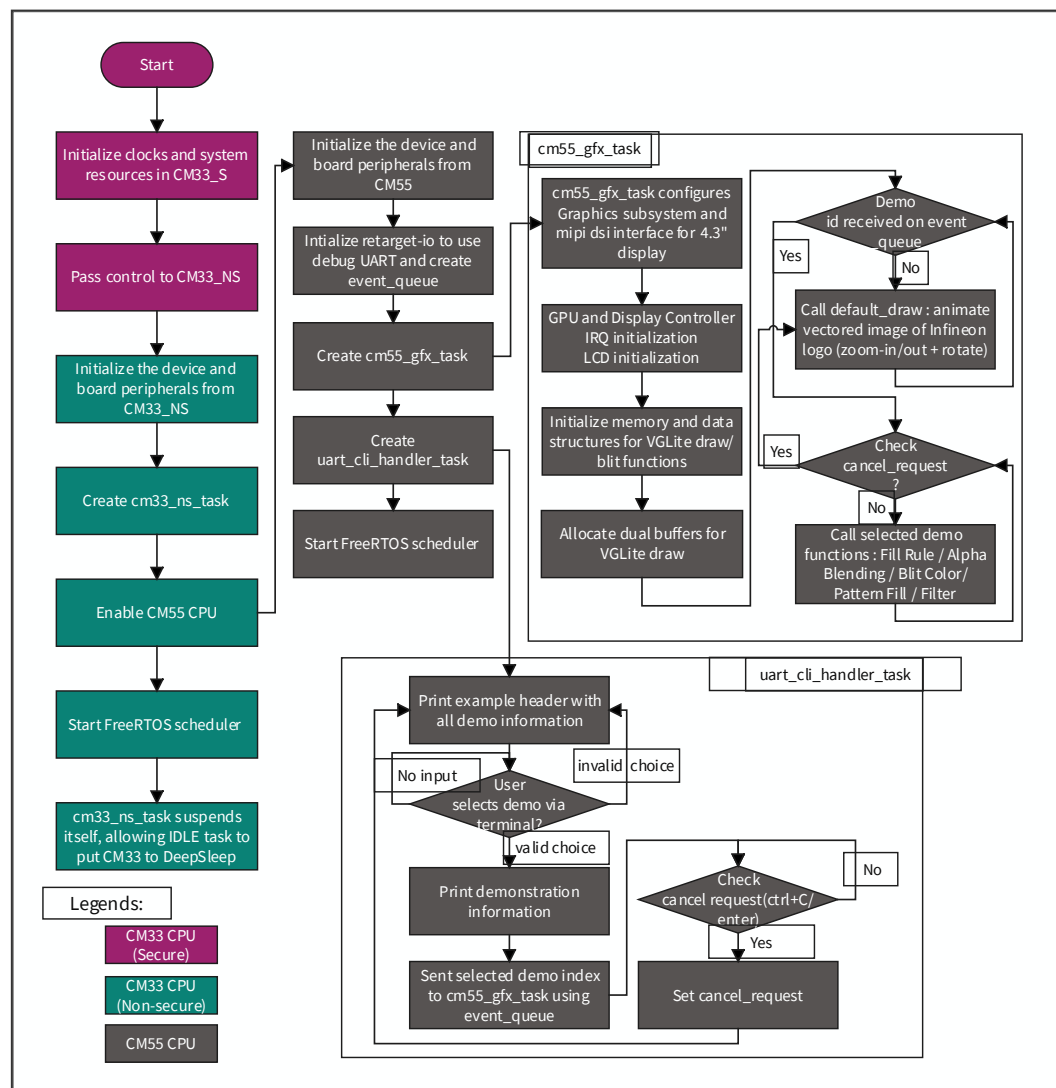
- Graphics processor unit (GPU)
 - Vivante® GCNanoUltraV
- Display controller
 - Vivante® DC8000Nano
- MIPI DSI host controller
 - Synopsys MIPI DSI host controller
 - Integrated D-PHY



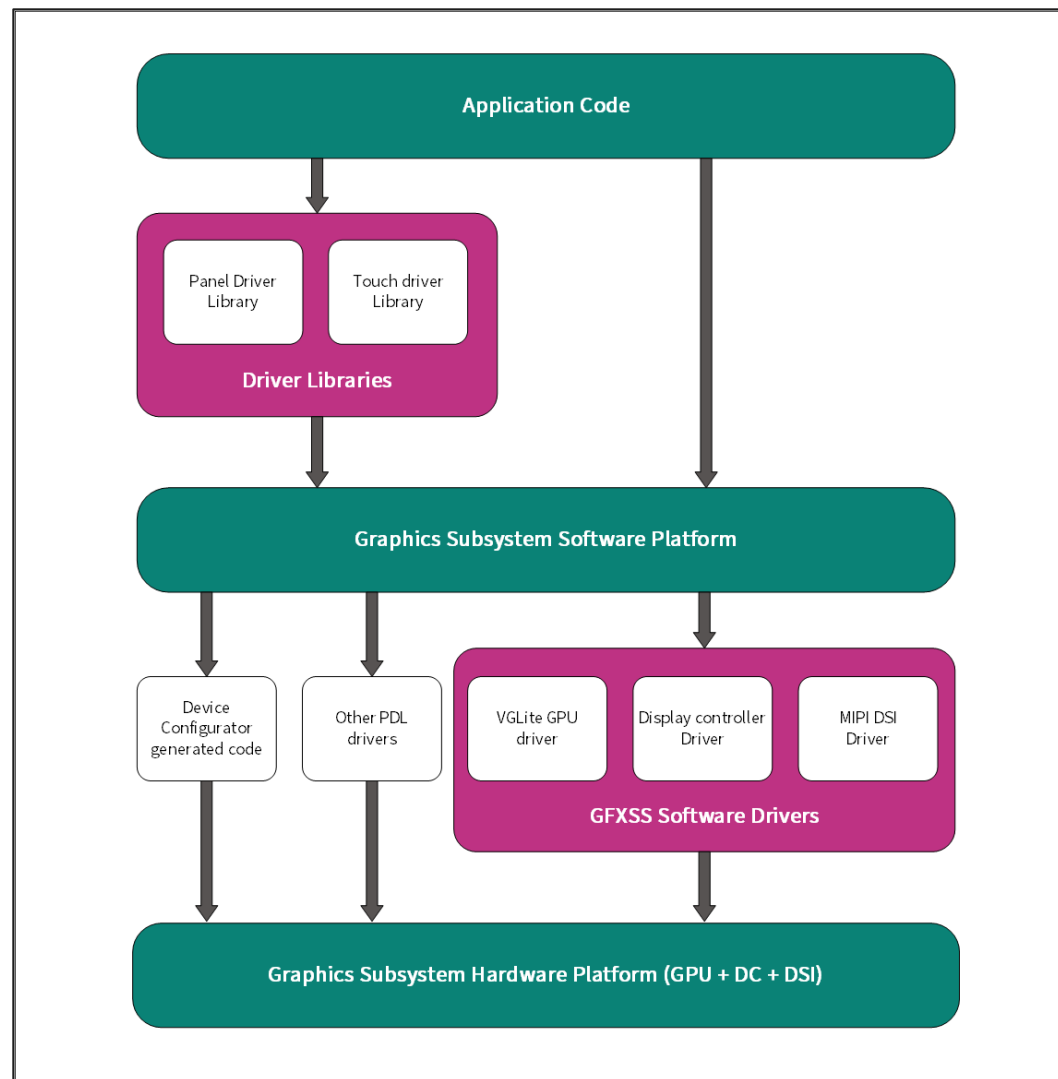
Graphics data flow



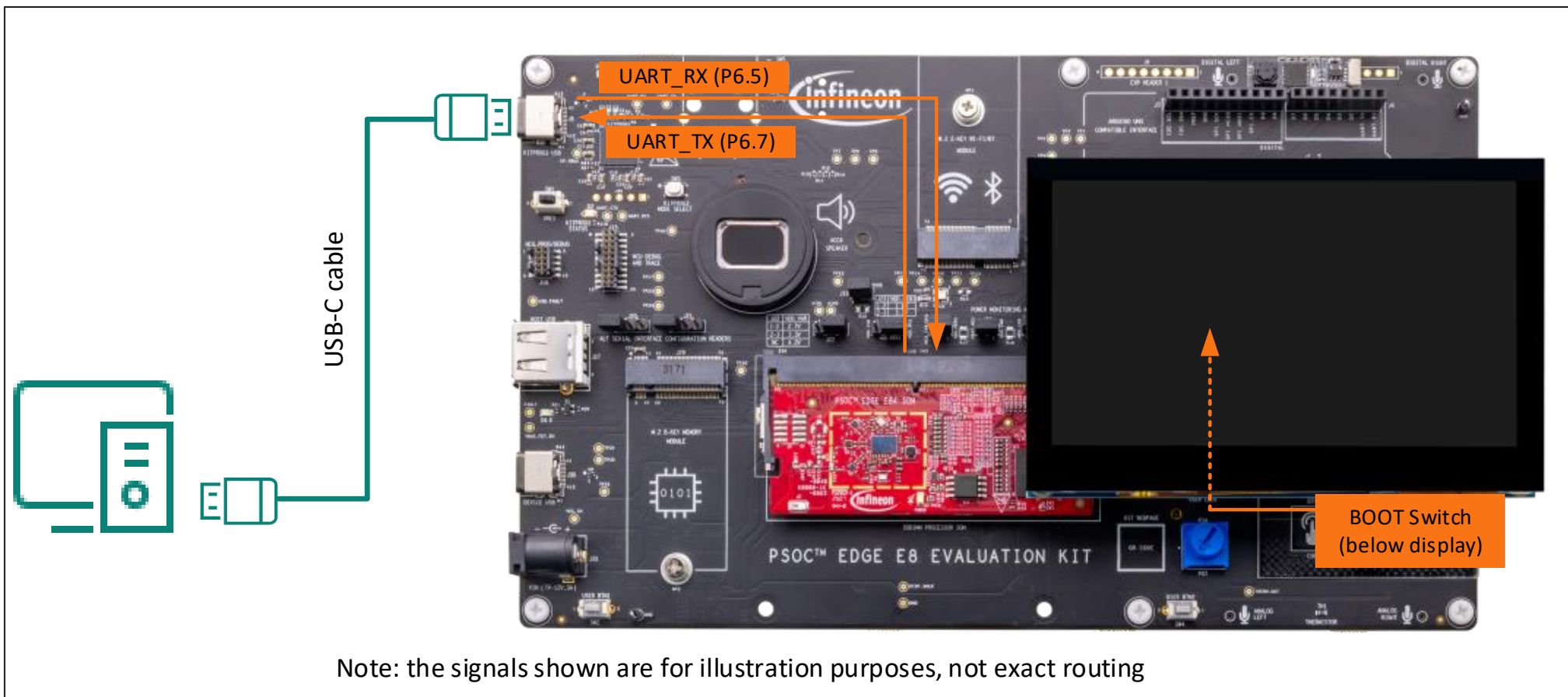
Graphics using VGLite code example: Flow chart



GFXSS software architecture used in VGLite code example



Hardware overview



Lab 4: Graphics using VGLite

What will you learn?

- How to use Vivante® platform-independent VGLite graphics API to carry out hardware-accelerated 2D vector drawing operations
- How to render images on the TFT LCD display
- How to render a custom image
- How to use VGLite APIs

Observe output on the PSoC™ Edge E84 MCU dev kit

```

COM41 - Tera Term VT
File Edit Setup Control Window Help
***** PSoC Edge MCU: Graphics using UGLite API *****

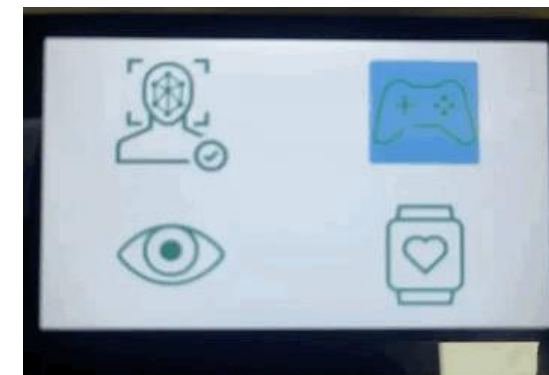
List of operations to choose from:
1. Fill Rules (Vector Paths)
2. Alpha Blending (Vector Paths)
3. Blit Color Rendering
4. Pattern Fill
5. UI/Filter Demo

FPS: 55.555 ! CPU usage: 1%
  
```

Default serial output

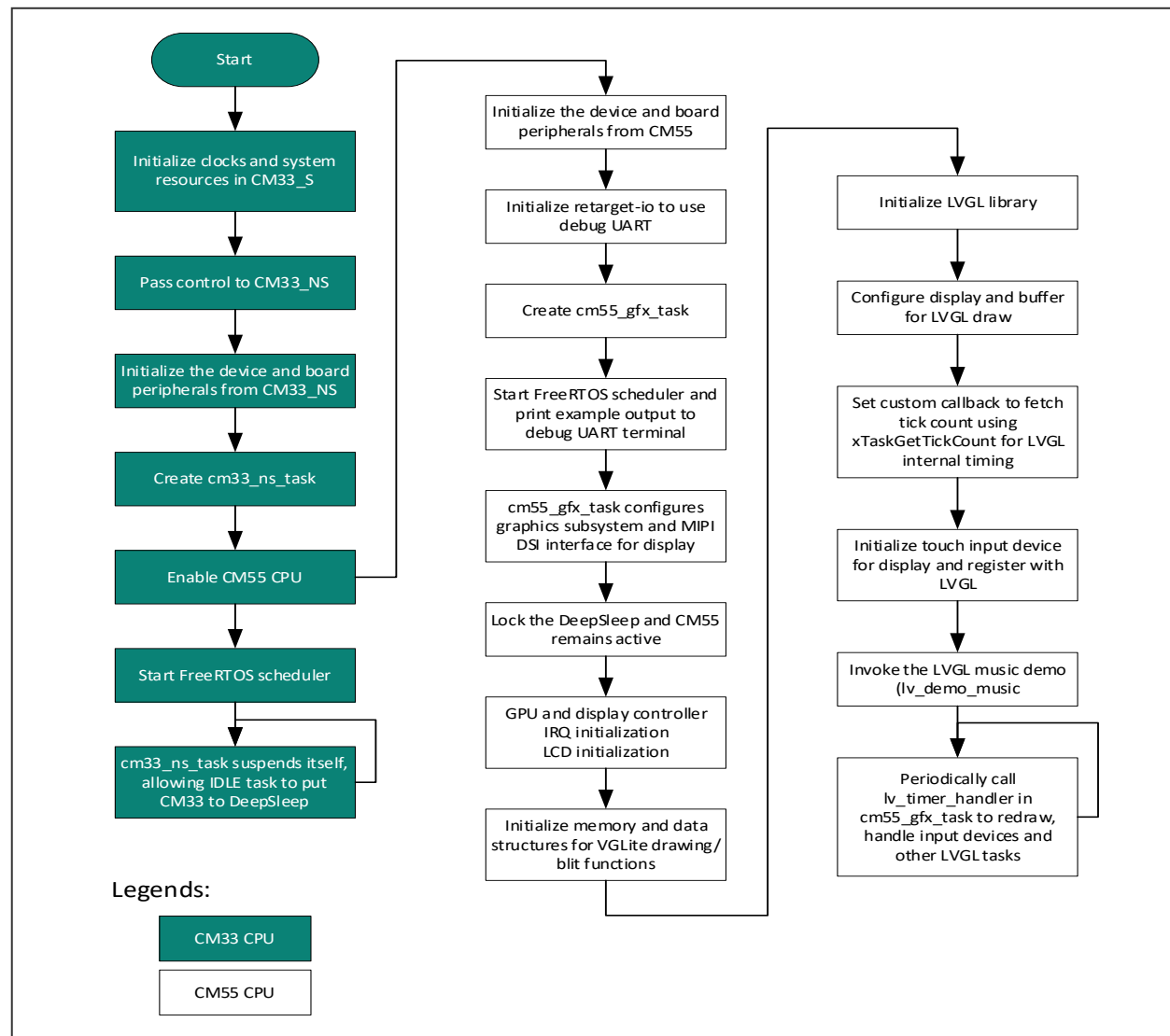


Display rendering image

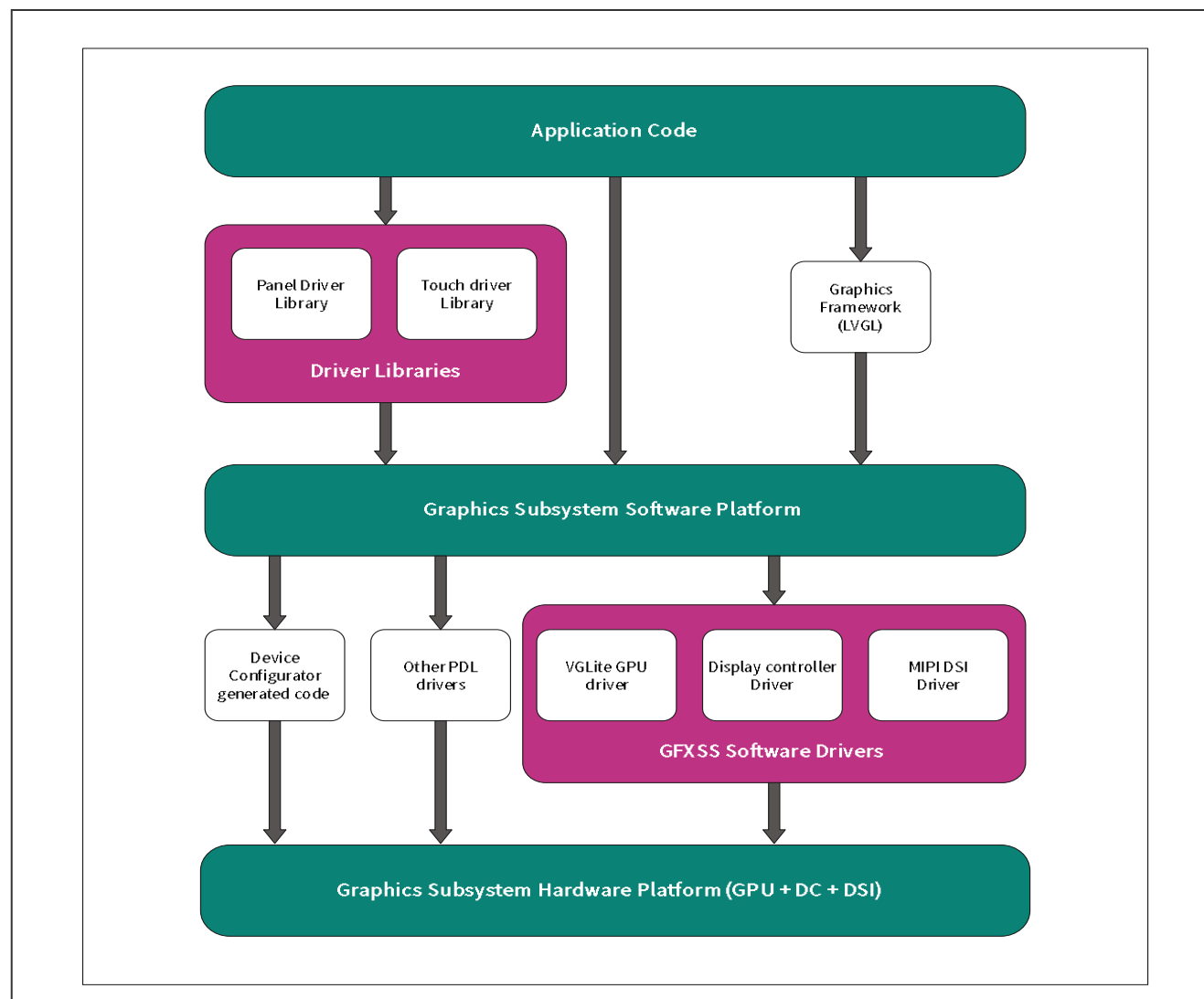


UI Filter demo

LVGL demo code example flow chart



GFXSS software architecture for LVGL code example



Lab 5: Graphics LVGL demo

What will you learn?

- How to run a demo application using the LVGL library
- How to use EEZ-Studio to generate a custom UI

Observe output on the PSoC™ Edge E84 MCU dev kit

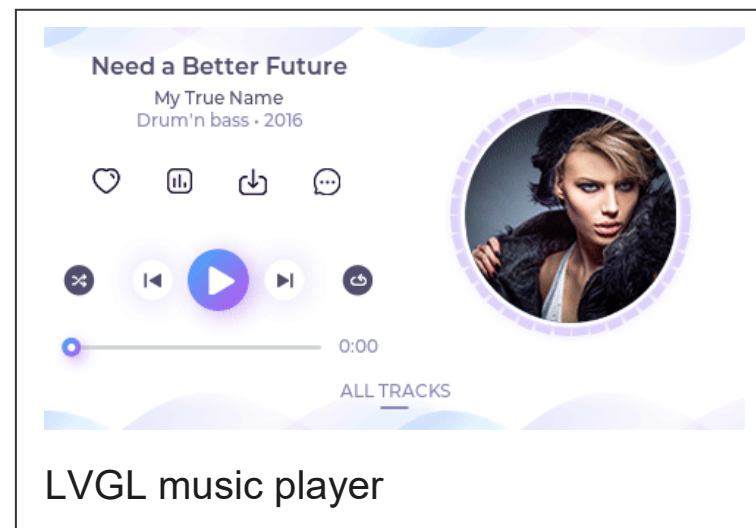
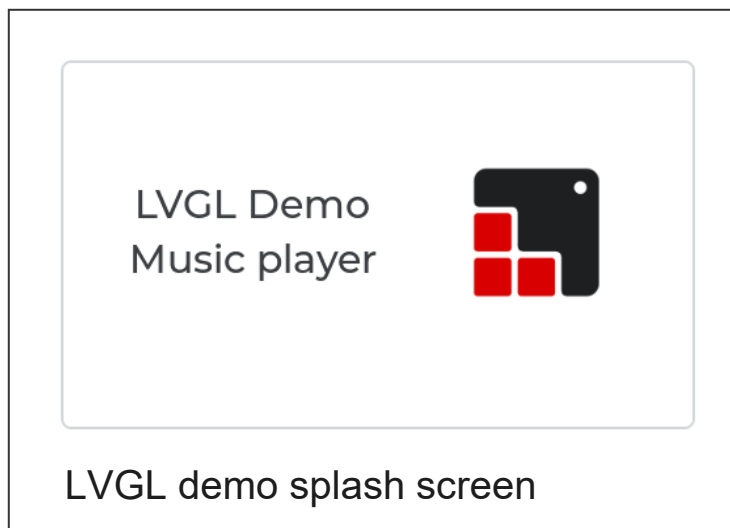
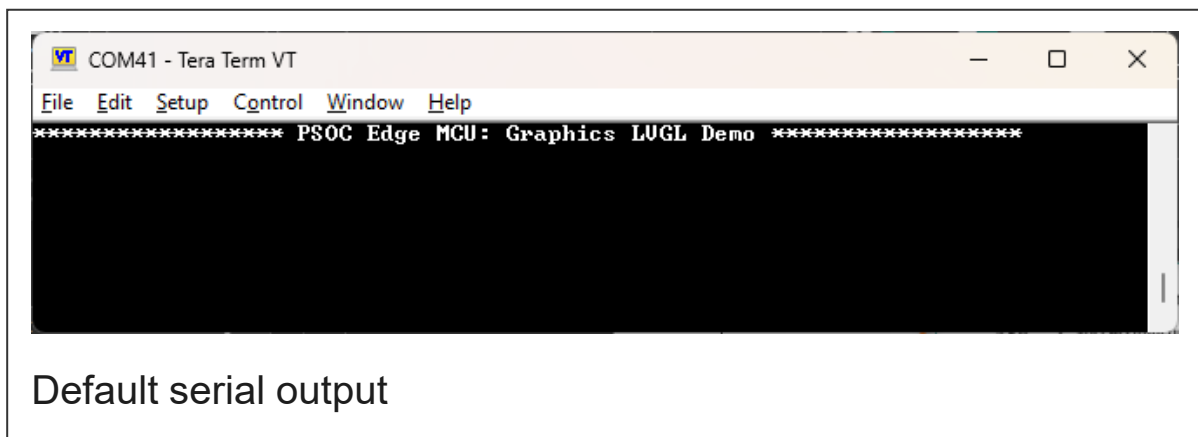
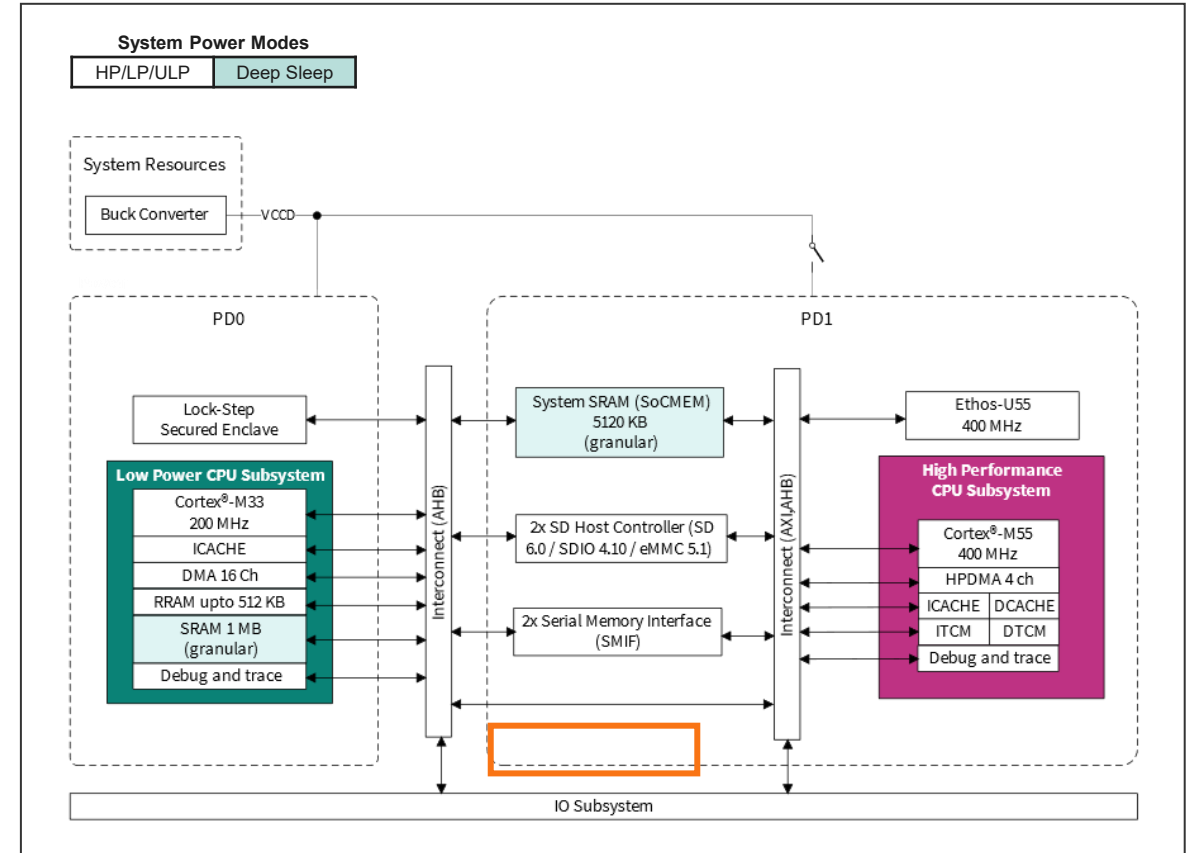


Table of contents

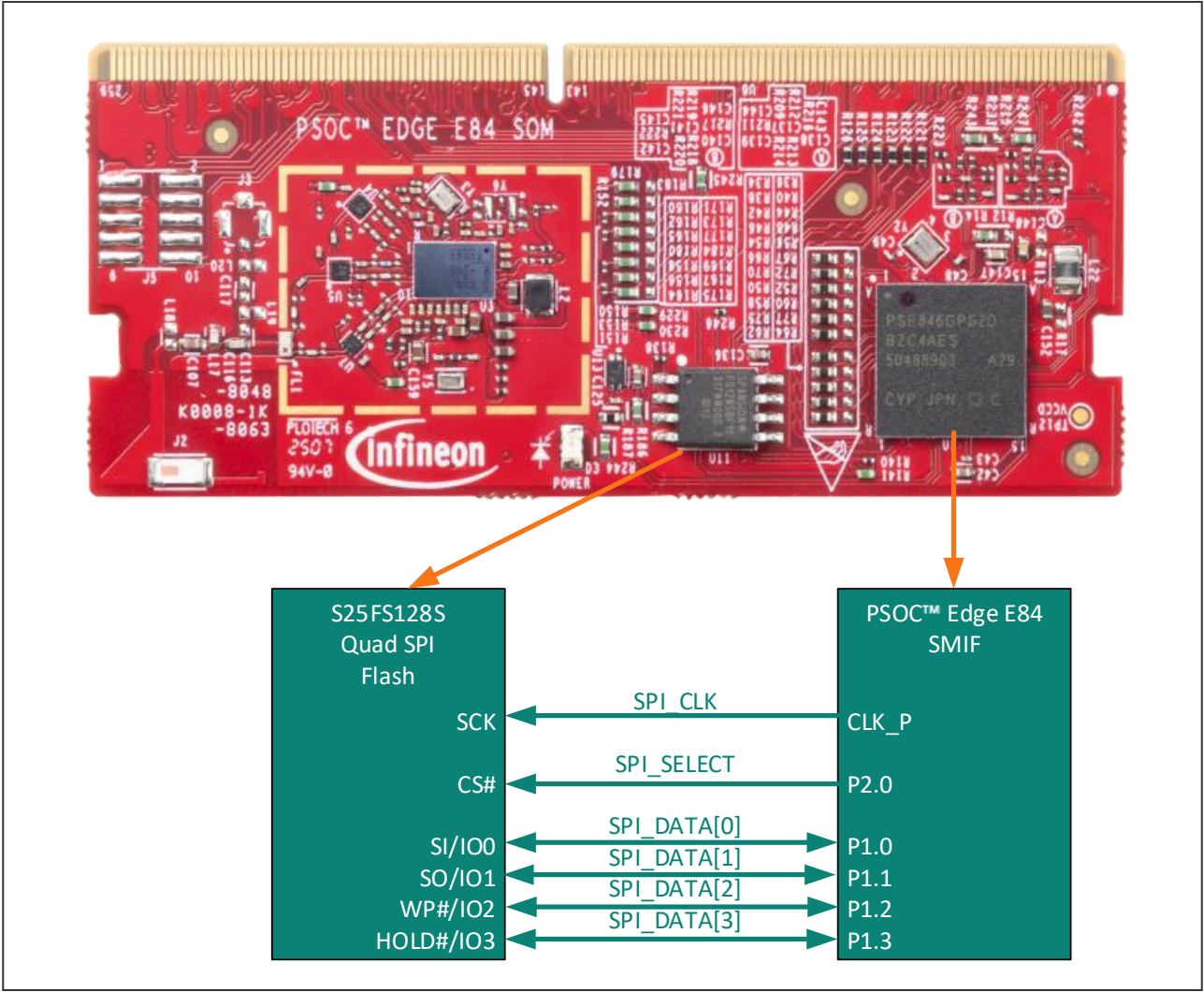
1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

Serial memory interface (SMIF)

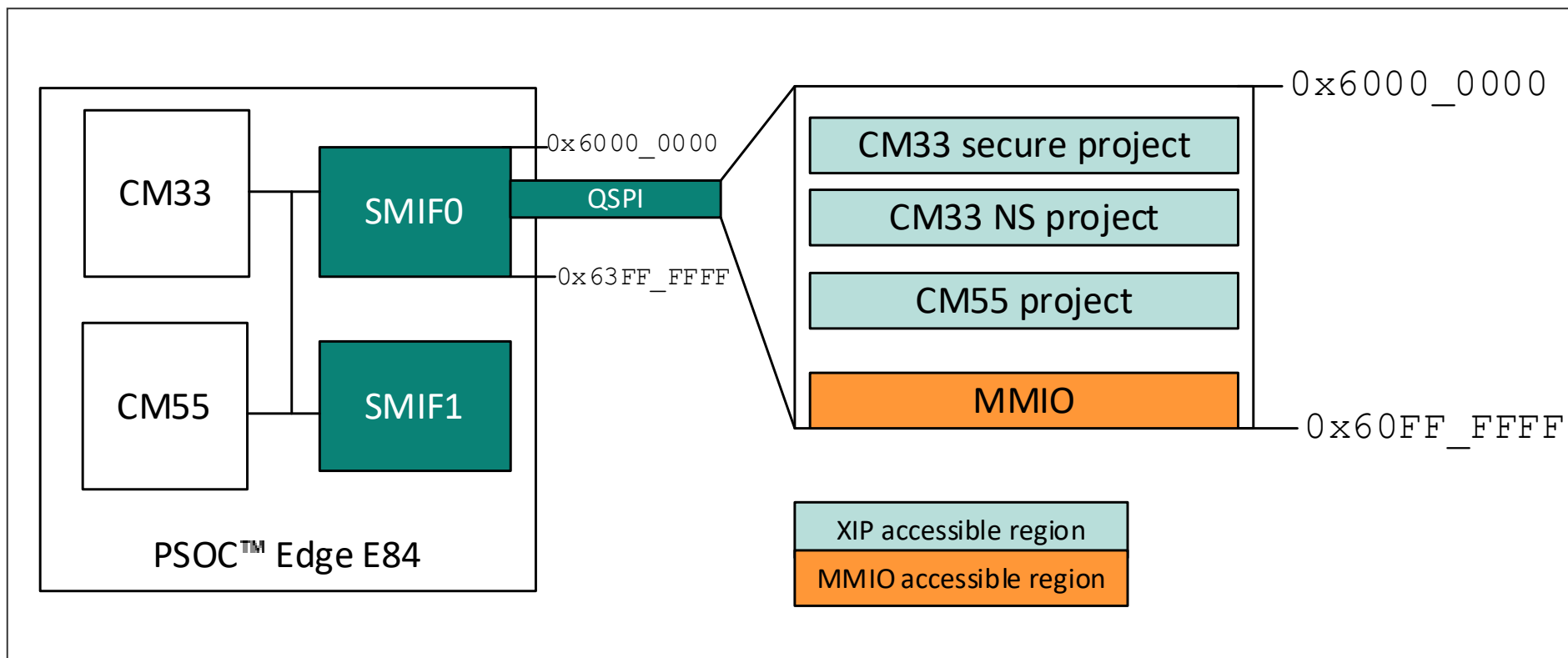
- SMIF interfaces
 - PSOC™ Edge has up to two SMIFs: SMIF0 and SMIF1
 - Availability is package-dependent
- Memory capacity
 - Each interface can access up to 1 GB of address space, or 64 MB in XIP mode
- External device support
 - Compatible with various SPI and HYPERBUS™ memory devices
 - Supports SDR and DDR modes
- Transfer modes
 - XIP for direct memory address execution
 - MMIO for flexibility in device operations
- Security
 - Features on the-fly encryption and decryption



Hardware diagram

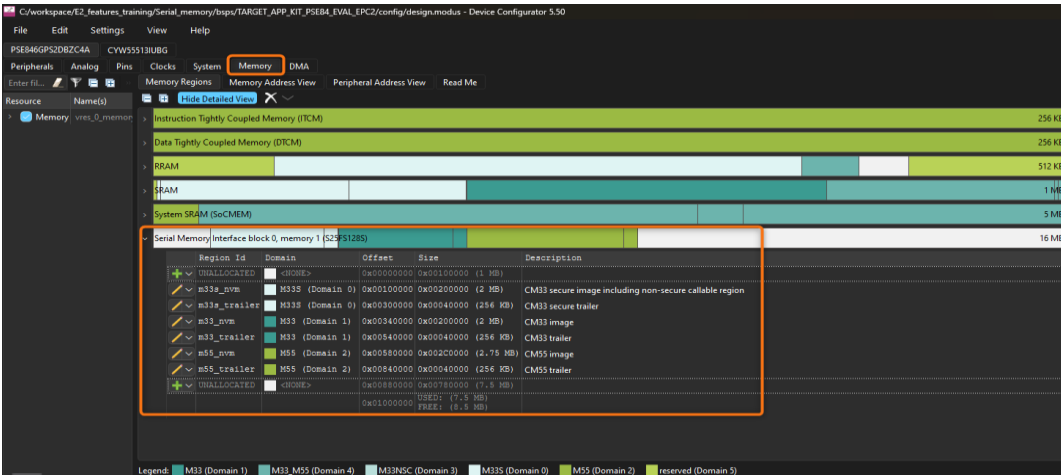


Serial flash read/write



Memory Configurator

- Available in ModusToolbox™ Device Configurator
- Allows to observe internal and external memory allocations and sections
- Easier memory customization
- Friendly visualization of memory maps for Cortex®-M33 (CM33) and Cortex®-M55 (CM55), and for secure and non-secure memories



The top screenshot shows the 'Memory' tab in the Device Configurator. It displays a hierarchical view of memory regions for the PSEB46GPS2DBZC4A device. The regions include:

- Instruction Tightly Coupled Memory (ITCM): 256 KB
- Data Tightly Coupled Memory (DTCM): 256 KB
- RAM: 512 KB
- SRAM: 1 MB
- System SRAM (SoCMEM): 5 MB
- Serial Memory Interface block 0, memory 1 (22951285): 16 MB

The bottom screenshot shows the 'Memory Address View' for Core CM33. It displays a detailed memory map with columns for Address, Memory Views, IDAU, SAU, IDAU / SAU, MPC, and Regions. The map shows the following regions:

Address	Memory Views	IDAU	SAU	IDAU / SAU	MPC	Regions
0x60100000	XIP_PORT0 16 MB Executable Writable Programmable Nonvolatile	NS 256 MB	NS 256 MB		M33S (Domain 0) PC2: RW	m33s_nvm (2 MB)
0x60300000						m33s_trailer (256 KB)
0x60340000						m33_nvm (2 MB)
0x60540000						m33_trailer (256 KB)
0x60580000						m55_nvm (2.75 MB)
0x60840000					M55 (Domain 2) PC2: RW	m55_trailer (256 KB)
0x60A00000						custom (1 MB)
0x60B00000						
0x61000000						
0x70000000		S 256 MB	NS 3 GB	S 256 MB		

Lab 6: Serial flash read/write

What will you learn?

- How to read, write and erase an external NOR memory using SMIF in MMIO mode
- How to use the Serial Flash Discoverable Parameters (SFDP) standard
- How to observe memory configuration in Memory Configurator
- How to place code or data at a desired location in external flash

Observe output on UART terminal

```
COM41 - Tera Term VT
File Edit Setup Control Window Help

1. Erasing 65536 bytes from offset address 0x7e0000
2. Reading after Erase & verifying that each byte is 0xFF
Received Data (64 bytes):
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
3. Writing data to offset address 0x7e0000
Written Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
4. Reading back for verification
Received Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
=====
SUCCESS: Read data matches with written data!
=====
```

1. Default application

```
COM41 - Tera Term VT
File Edit Setup Control Window Help

1. Erasing 65536 bytes from offset address 0xb00000
2. Reading after Erase & verifying that each byte is 0xFF
Received Data (64 bytes):
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
3. Writing data to offset address 0xb00000
Written Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
4. Reading back for verification
Received Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
=====
SUCCESS: Read data matches with written data!
=====
```

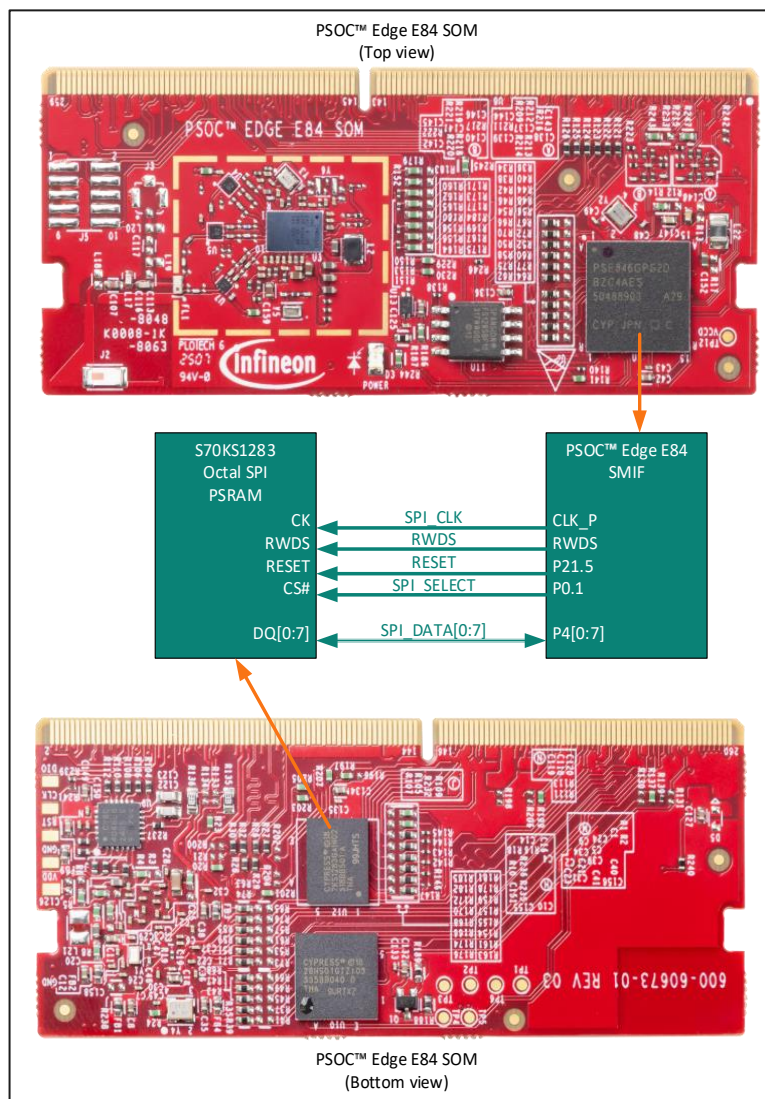
2. First modification

```
COM41 - Tera Term VT
File Edit Setup Control Window Help

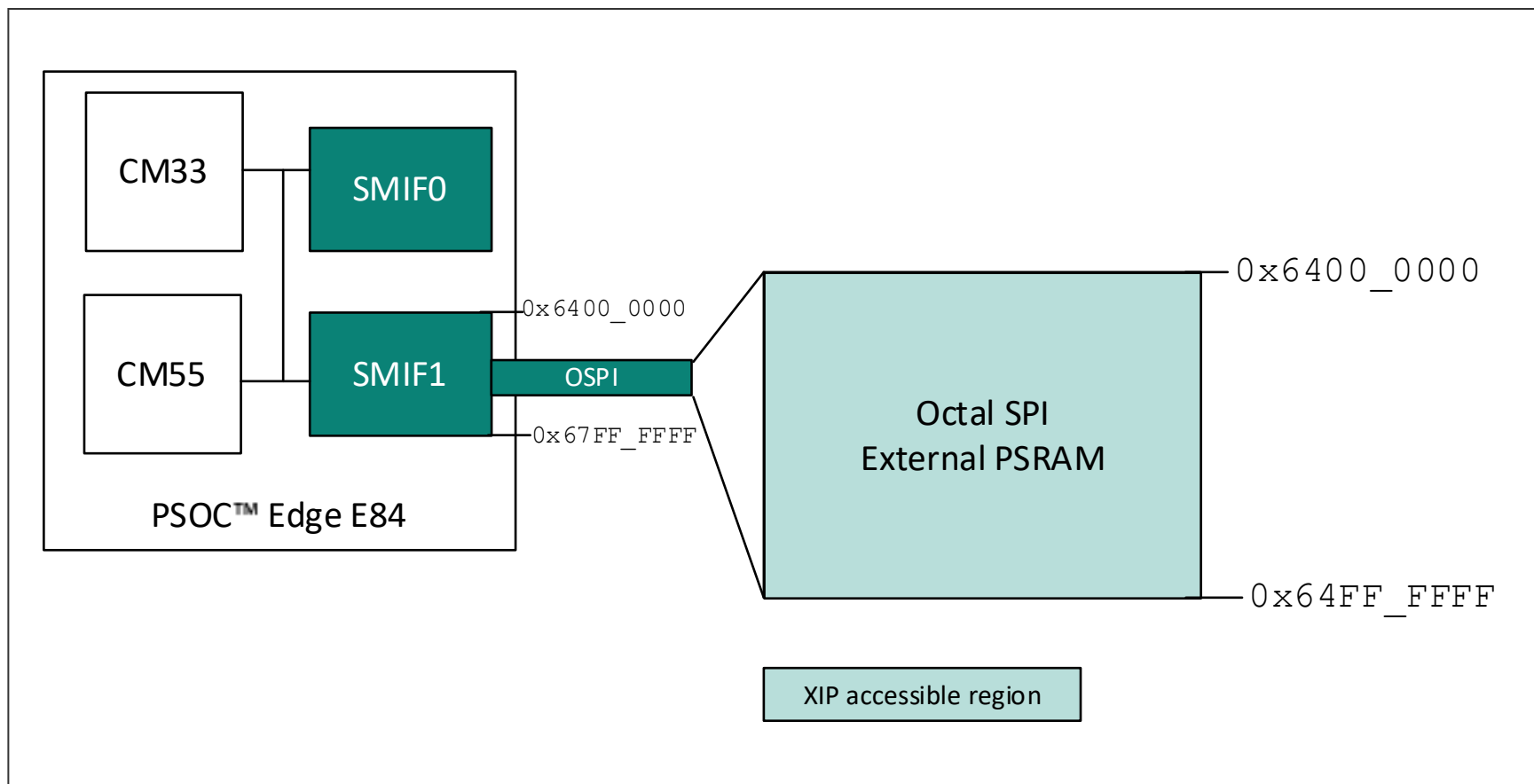
***** PSOC Edge MCU: Serial Flash Read and Write Test*****
Total Flash Size: 16777216 bytes
1. Erasing 65536 bytes from offset address 0xb00000
2. Reading after Erase & verifying that each byte is 0xFF
Received Data (64 bytes):
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF
3. Writing data to offset address 0xb00000
Written Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
4. Reading back for verification
Received Data (64 bytes):
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F
0x10 0x11 0x12 0x13 0x14 0x15 0x16 0x17 0x18 0x19 0x1A 0x1B 0x1C 0x1D 0x1E 0x1F
0x20 0x21 0x22 0x23 0x24 0x25 0x26 0x27 0x28 0x29 0x2A 0x2B 0x2C 0x2D 0x2E 0x2F
0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x3A 0x3B 0x3C 0x3D 0x3E 0x3F
String in the external memory at address: 0x60a00018
Hello string from the custom data section!
Function call from external memory address: 0x60a00061
Hello from function stored in the custom code section!
=====
SUCCESS: Read data matches with written data!
=====
```

3. Second modification

PSRAM read/write in XIP mode



PSRAM read/write in XIP mode (contd.)



Lab 7: PSRAM read/write in XIP mode

What will you learn?

- How to interface an external PSRAM with the SMIF in XIP mode
- How to enable cache and when to use it
- How to place an array of data at a desired location of PSRAM

Observe output on a UART terminal

```
COM41 - Tera Term VT
File Edit Setup Control Window Help
***** PSOC Edge MCU: PSRAM Read and Write in eXecute-In-Place (XIP) mode*****
Cache is Disabled

***** Memory read write demo *****
Test data <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Destination data after memory copy <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Memory compare successful: Data matches test data
```

1. Default application

```
COM41 - Tera Term VT
File Edit Setup Control Window Help
***** PSOC Edge MCU: PSRAM Read and Write in eXecute-In-Place (XIP) mode*****
Cache is Enabled

***** Memory cache clean demo *****
Test data <16 bytes>:
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55

Data read before cache clean <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Memory compare failed: Data mismatch. Expected data and read data are different
Perform cache clean operation

Data read after cache clean <16 bytes>:
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55

Memory compare successful: Data matches test data

***** Memory cache invalidate demo *****
Test data <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Data read before cache invalidate <16 bytes>:
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

Memory compare failed: Data mismatch. Expected data and read data are different
Perform cache invalidate

Data read after cache invalidate <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Memory compare successful: Data matches test data
```

2. First modification

```
COM41 - Tera Term VT
File Edit Setup Control Window Help
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55
0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55

Memory compare successful: Data matches test data

***** Memory cache invalidate demo *****
Test data <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Data read before cache invalidate <16 bytes>:
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

Memory compare failed: Data mismatch. Expected data and read data are different
Perform cache invalidate

Data read after cache invalidate <16 bytes>:
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA
0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA 0xAA

Memory compare successful: Data matches test data

Modification
=====
Address of data_array = 0x74000000
data_array is populated with sequential numbers

Read Data <16 bytes>:
0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07
0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F

Address of data_array = 0x74000000
Each element of data_array is incremented by 1

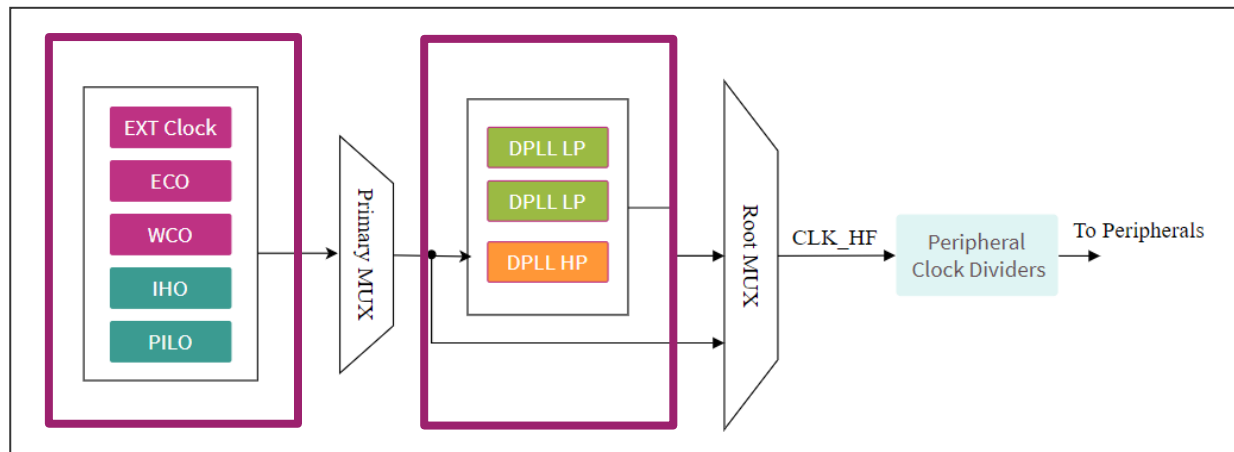
Read Data <16 bytes>:
0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08
0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F 0x10
```

3. Second modification

Table of contents

1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

Clocking system



Two internal clock sources

- Internal high-speed oscillator (IHO):
50 MHz $\pm 1\%$
- Precision internal low-speed oscillator (PILO):
 - 32.768 kHz $\pm 2\%$ (factory trim)
 - 32.768 kHz ± 250 ppm (with calibration)

Three external clock sources

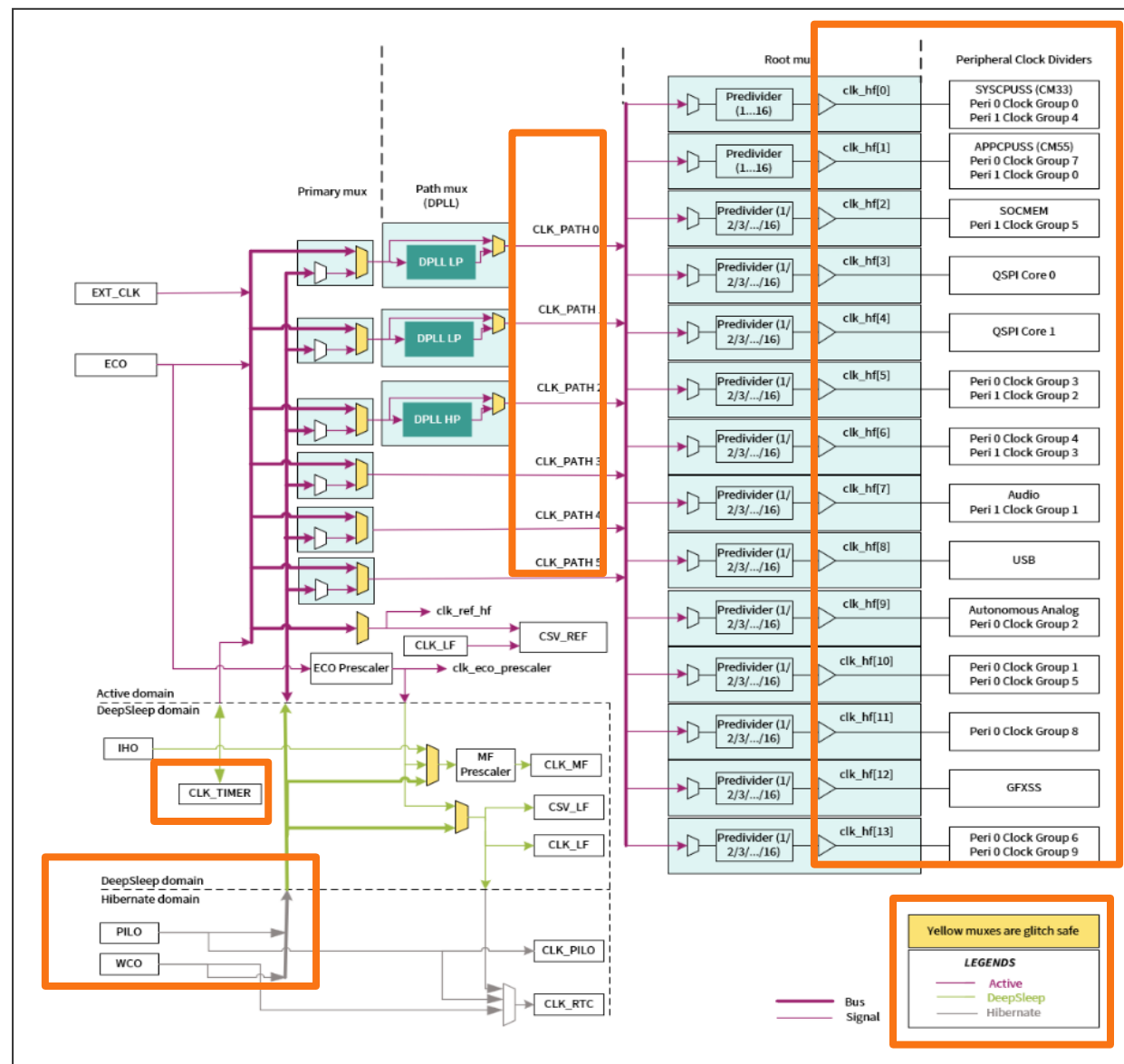
- External crystal oscillator (ECO):
4 - 38 MHz
- Watch crystal oscillator (WCO):
32.768 kHz
- External clock input: up to 100 MHz

Three digital phase-locked loops (DPLL)

- Two lower-frequency low power (DPLL LP): up to 500 MHz
- One high-frequency high performance (DPLL HP): up to 500 MHz

Clock tree

- Six path clocks
 - CLK_PATH0.... CLK_PATH5
- Fourteen high-frequency root clocks (CLK_HF)
 - CLK_HF0....CLK_HF13
- Low-frequency clock options
 - PILO, WCO and ECO
- Timer clock: fixed 1 MHz derived from IHO
- Peripheral clock dividers
 - Fourteen 8-bit integer
 - Five 16-bit integer
 - Six 16.5-bit fractional (16 integer bits, five fractional bits)
 - One 24.5-bit fractional (24 integer bits, five fractional bits)



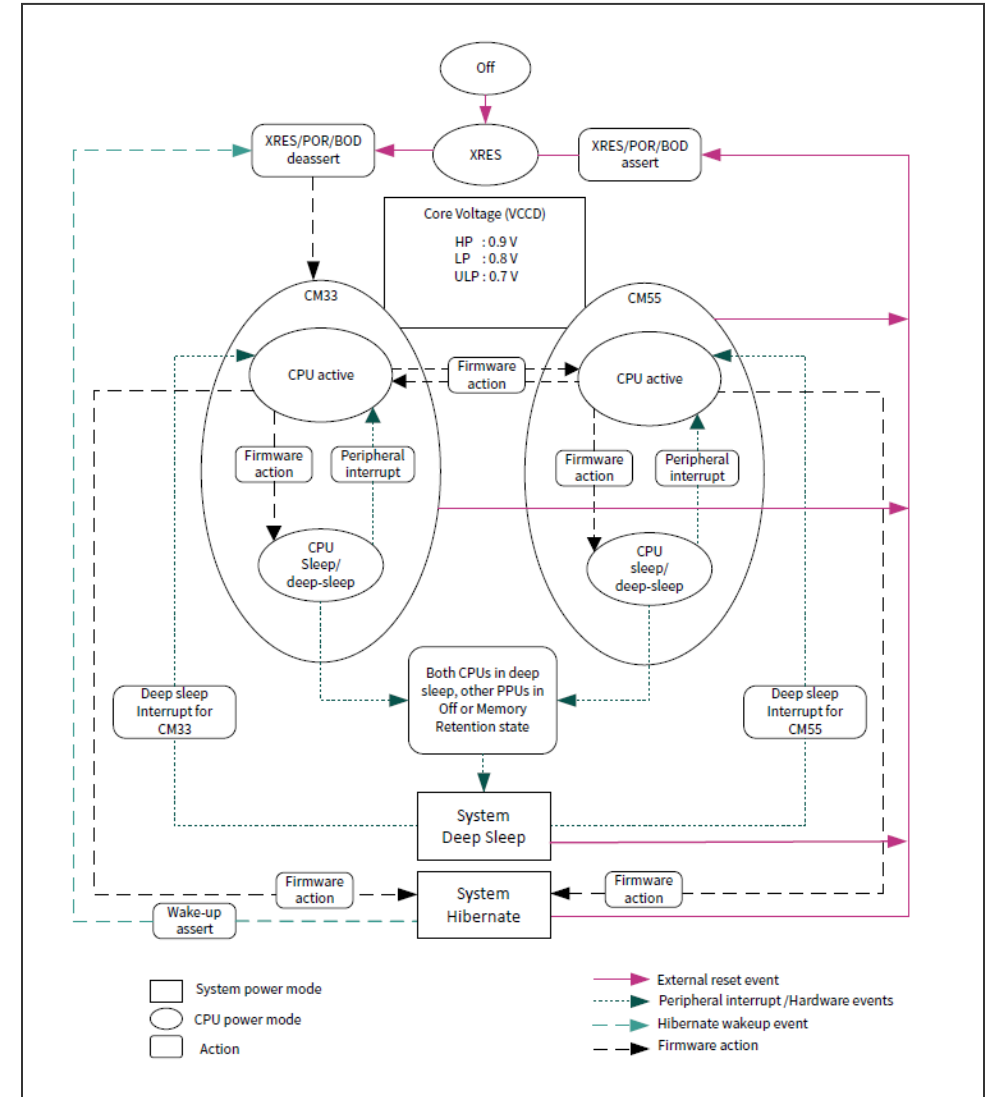
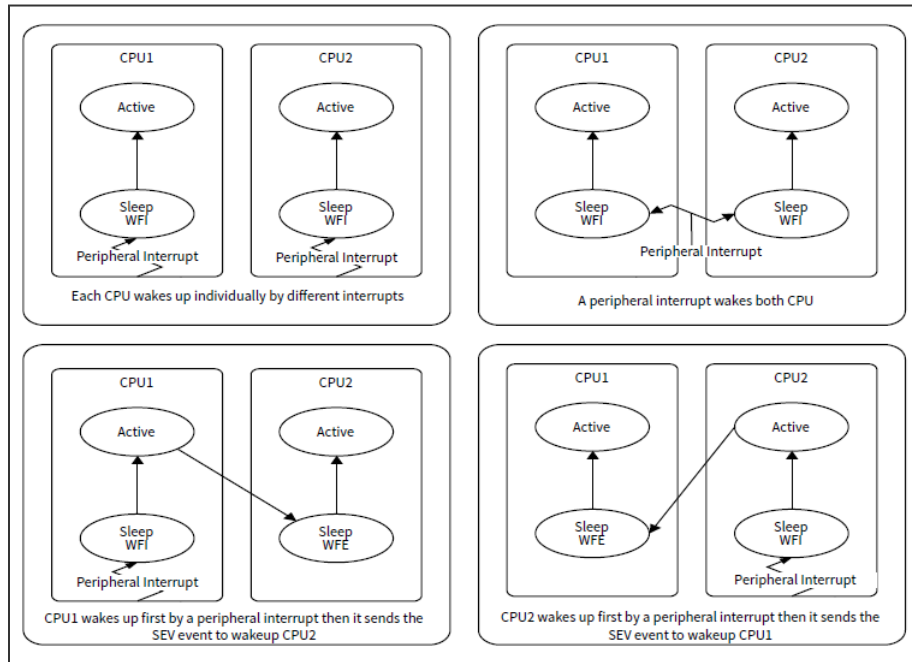
Device power modes

- Five system power modes
 - High-Performance (HP)
 - Low-Power (LP)
 - Ultra-Low-Power (ULP)
 - System Deep Sleep
 - Deep Sleep RAM and Deep Sleep Off
 - System Hibernate
- Three CPU power modes
 - CPU Active
 - M33: 73 μ W/MHz at 200 MHz
 - M55 : 131 μ W/MHz at 400 MHz
 - CPU Sleep
 - M33: 38 μ W/MHz
 - M55: 78 μ W/MHz
 - CPU Deep Sleep
 - SRAM retention from 64 KB to 6.5 MB
 - SRAM granularity 64 KB

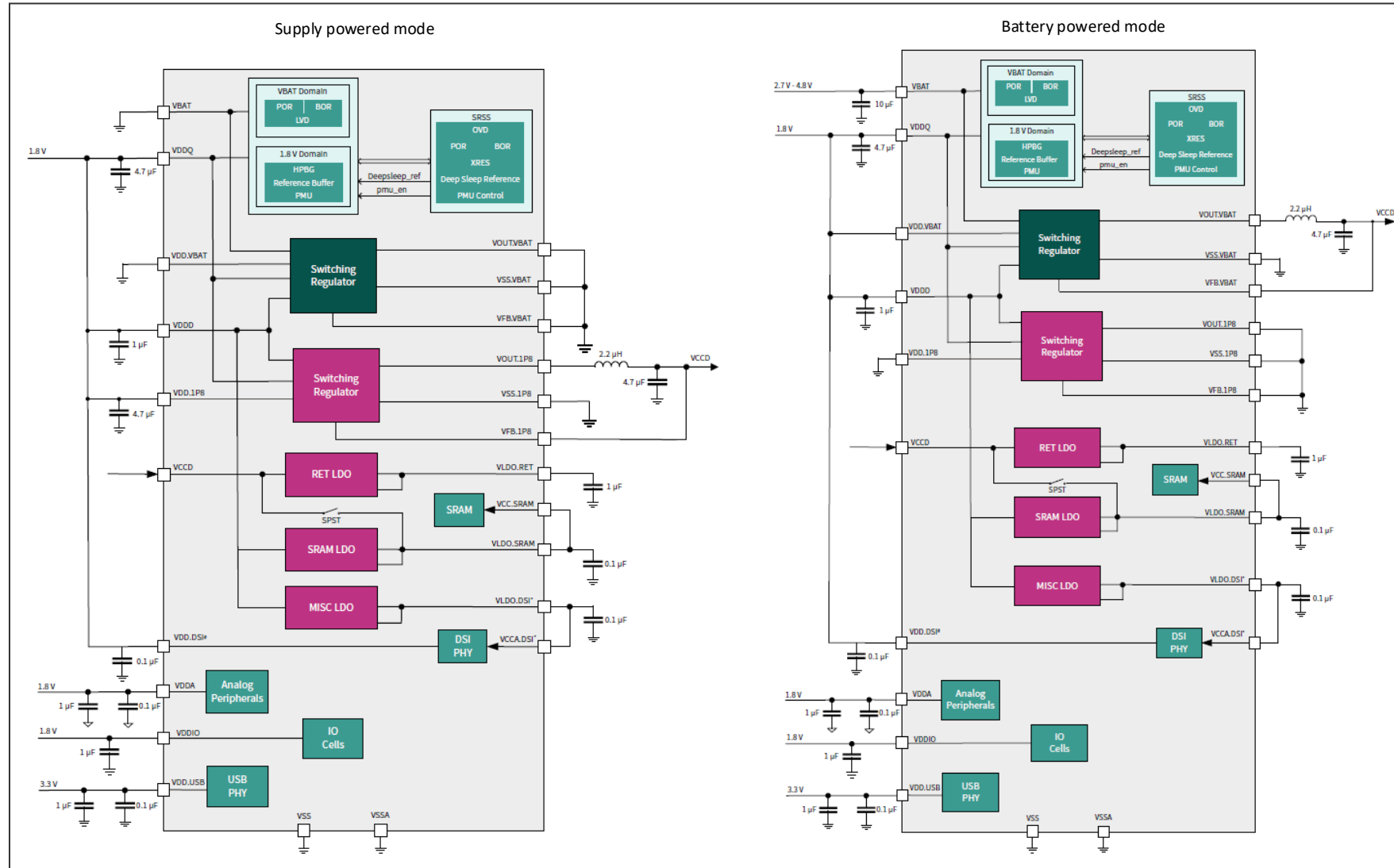
System power mode	CPU core voltage	Max frequency Cortex®-M33	Max frequency Cortex®-M55
System High-Performance (HP)	0.9 V	200 MHz*	400 MHz
System Low-Power (LP)	0.8 V	70 MHz	140 MHz*
System Ultra-Low-Power (ULP)	0.7 V	50 MHz	50 MHz
System Deep Sleep	0.7 V	Clock gated	Clock gated
System Hibernate	Off	Off	Off

Power mode transitions

- CM33 and CM55 can go in different CPU power modes independently
- System Deep Sleep mode is achieved when the CM33 and CM55 both enter their CPU Deep Sleep modes
- Core voltage affects both cores

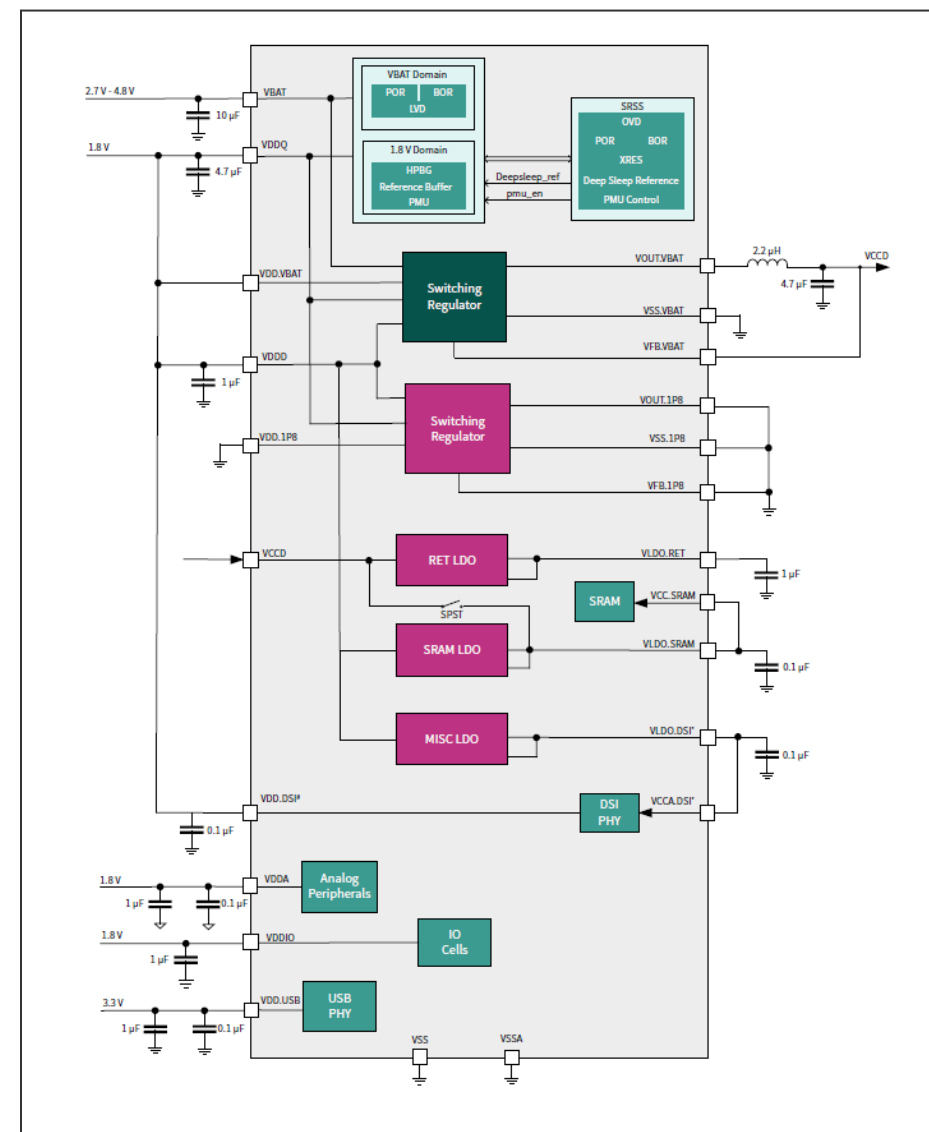


Power supply options



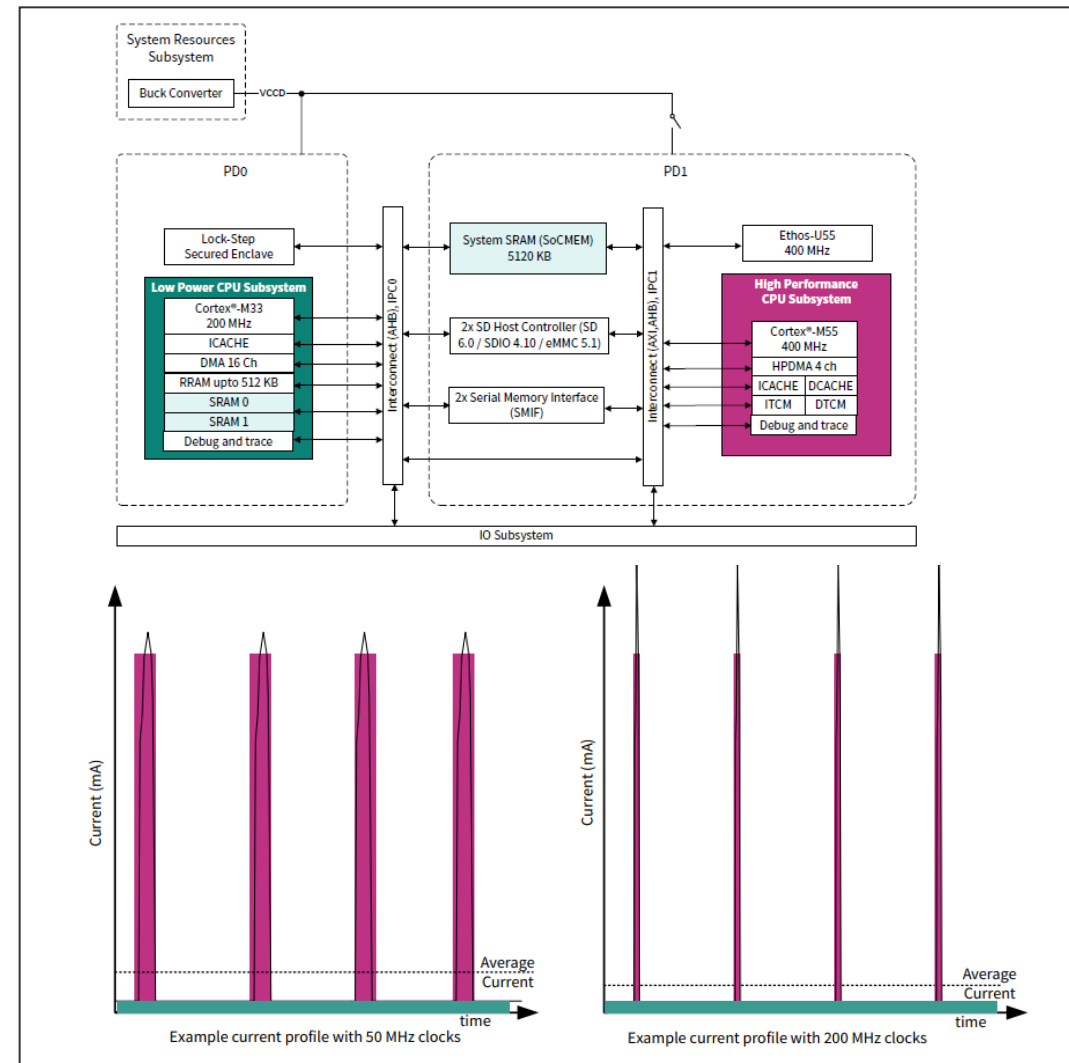
Power management

- Operating voltage range:
 - VBAT : 2.7 V to 4.8 V
 - VDDD: 1.8 V $\pm$ 5%
- User-selectable core logic operation:
 - System High-Performance (HP): Core logic runs at 0.9 V
 - System Low-Power (LP): Core logic runs at 0.8 V
 - System Ultra-Low-Power (ULP): Core logic runs at 0.7 V
- Multiple on-chip regulators
 - Two SISO buck converter
 - Multiple low power regulators
 - RET_LDO, SRAM_LDO and MISC_LDO
- Two POR circuits to monitor VDD and VBAT
- Three BOD circuits to monitor
 - VDDD, VCCD, and VBAT
- One LVD circuit to monitor VBAT
- One OVD block monitoring VCCD

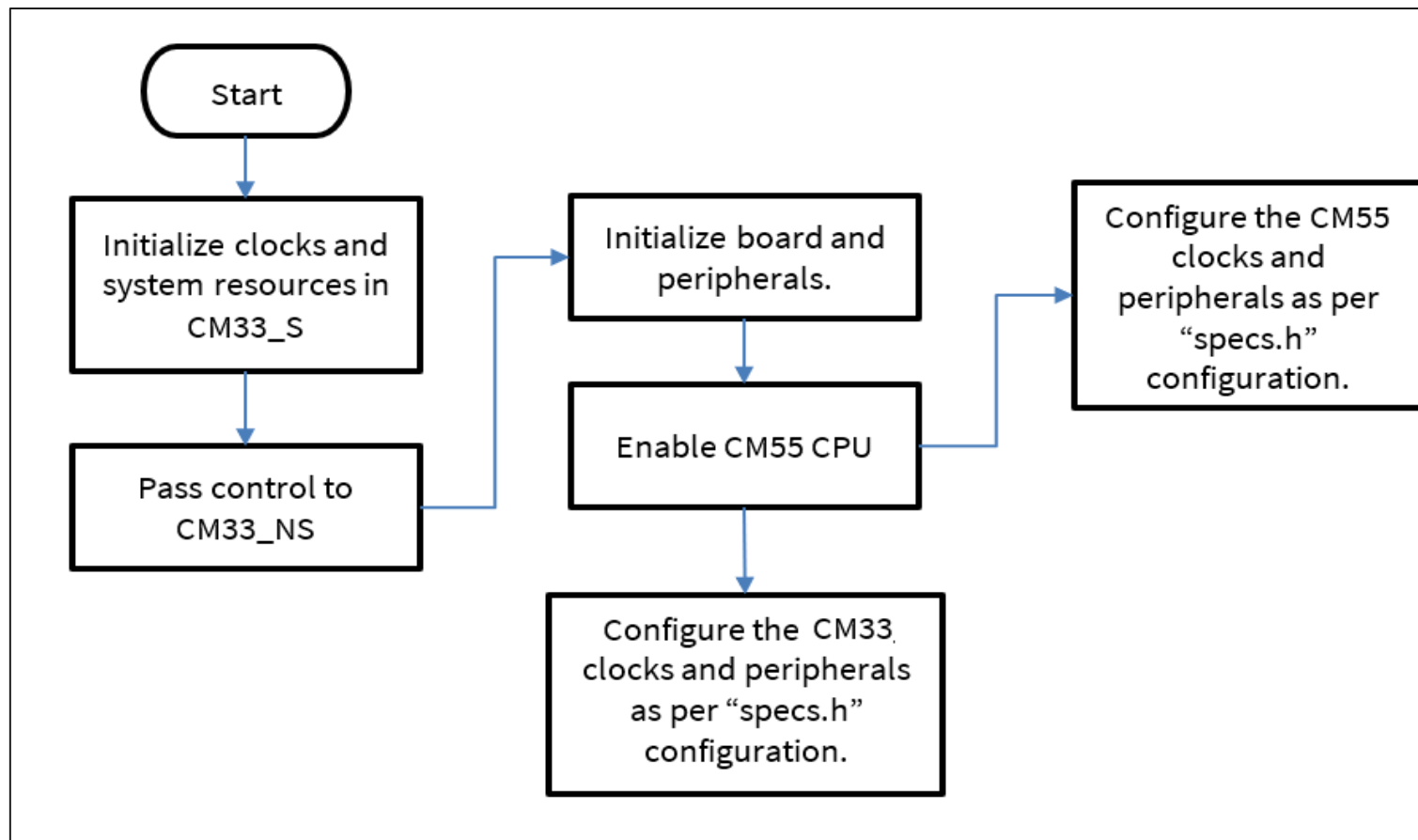


Power saving techniques

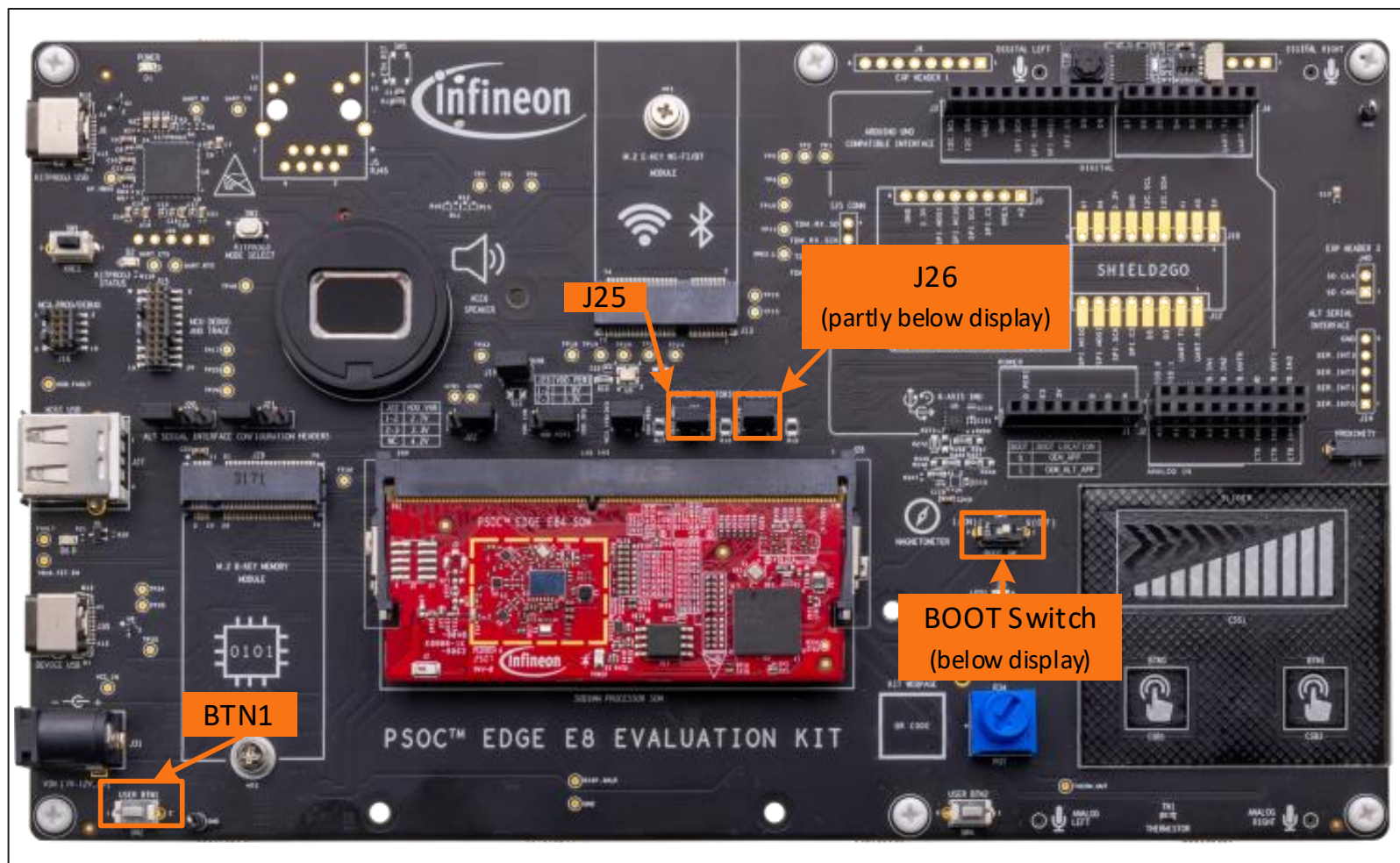
- Turning off PD1 domain
- Disabling CPUs
- Optimize clock configuration
- Leverage periodic wake-up timers
- Utilize tick less timer in FreeRTOS
- Use DMA to move data
- Split tasks between CPUs
- Use GPIOs to gate current paths
- Disable unused peripherals and clocks
- Terminate GPIOs properly
- Turn off SRAM banks
- Leverage co-processors and accelerators: Ethos™-U55, NNLite, Helium™ DSP, crypto
- Leverage low power features of peripherals: audio subsystem, SCB, analog subsystem, graphics, and so on



Power measurements



Power measurements in EVK



Lab 8: Power measurements

What will you learn?

- How to configure and run benchmarks in various PSOC™ Edge E84 MCU power modes such as Active, System Deep Sleep and System Hibernate
- How to measure the current consumed by the PSOC™ Edge E84 MCU

Observe power consumption in different modes



SIDH00A: Sleep



SIDHIBA: Hibernate

Lab 9: Switching power modes

What will you learn?

- How to transition between different system power modes:
High-Performance, Low-Power, Ultra-Low-Power, Deep Sleep
- How to measure the current consumed by the PSOC™ Edge E84 MCU

Observe power consumption in different modes

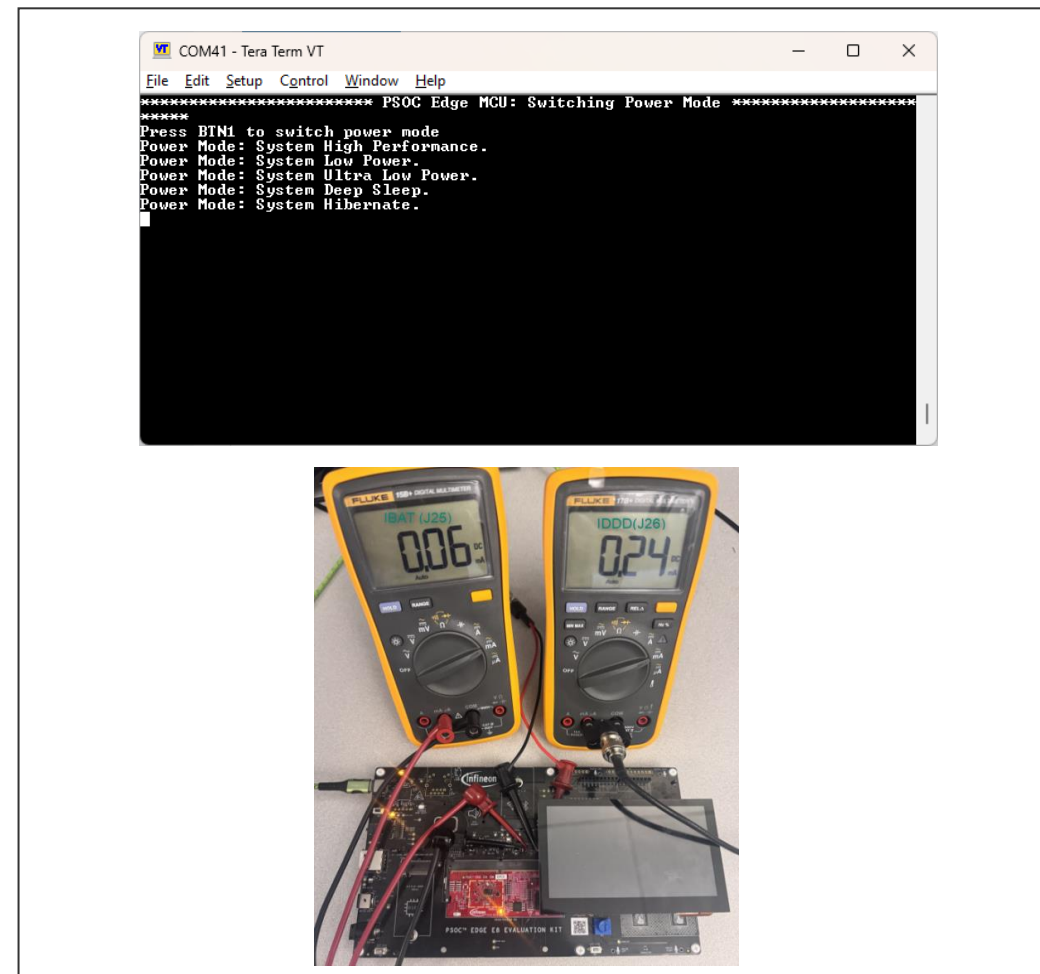
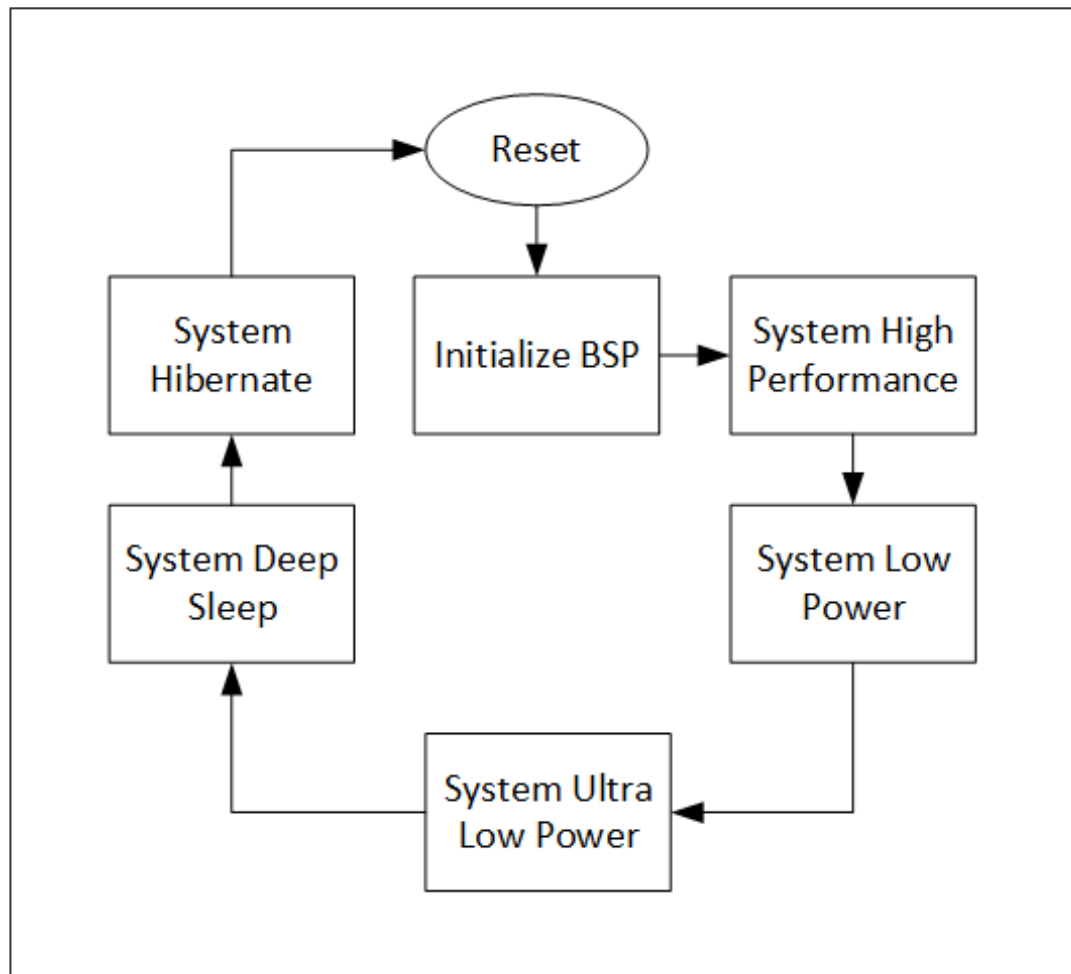


Table of contents

1	Audio	5
2	SensorHub	17
3	Graphics	24
4	External memory (SMIF)	36
5	Power modes	47
6	AI/ML using DEEPCRAFT™ and ModusToolbox™	61

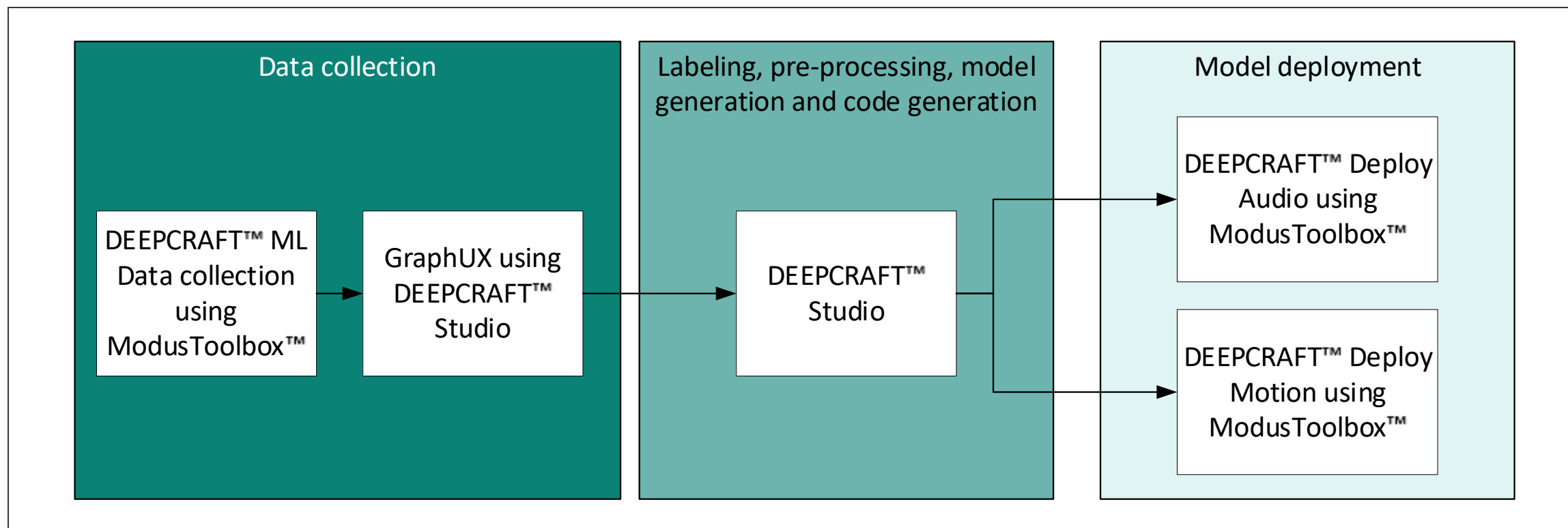
PSOC™ Edge ML acceleration

- CM55 includes Ethos™-U55 NPU running up to 400 MHz
 - Ethos™-U55 supports int8 operations
 - Float models run on CM55 only
- CM33 includes the NNLite block
 - NNLite being an Infineon IP NPU
 - Supports int8 operations
- Use of an NPU also involves use of the corresponding CPU
- Performance is very dependent on model architecture as shown in the tables

IMU Based Model			
Core	Core Frequency(MHz)	Inference Time(ms)	Cycle count
CM4 – Float	150	15.268	2,290,200
CM4 – Int8	150	4.759	713,850
M33 – Float	200	12.436	2,487,200
M33 – Int8	200	4.666	933,200
M55 – Float	400	4.590	1,836,000
M55 + U55 – Int8	400	0.058	23,200

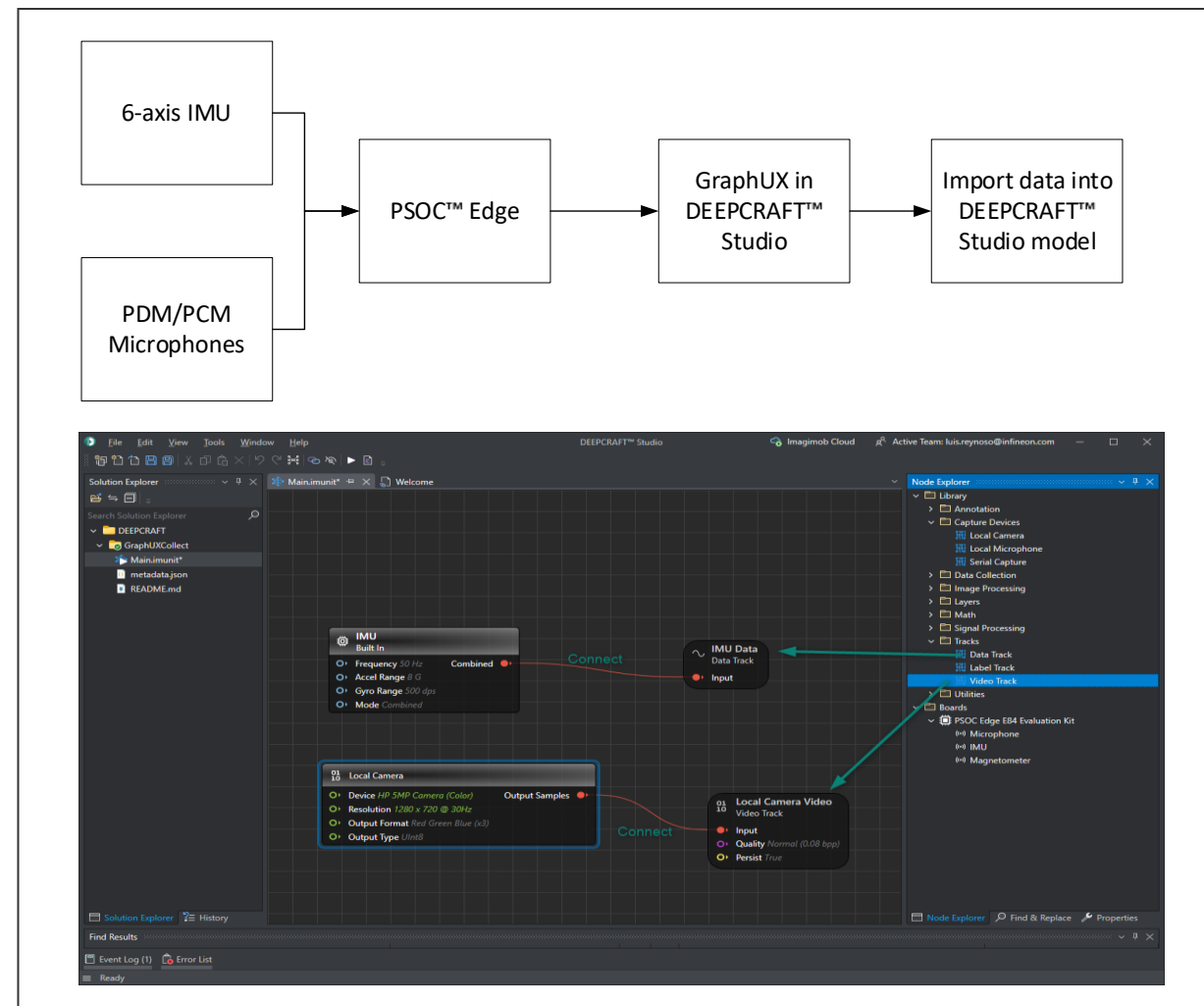
PDM Based Model			
Core	Core Frequency(Hz)	Inference Time(ms)	Cycle count
CM4 - Float	150	111.643	16,746,450
CM4 – Int8	150	30.459	4,568,850
M33 – Float	200	82.409	16,481,800
M33 – Int8	200	25.999	5,199,800
M55 – Float	400	33.180	13,272,00
M55 + U55 – Int8	400	0.280	112,000

ML pipeline with DEEPCRAFT™ Studio and ModusToolbox™

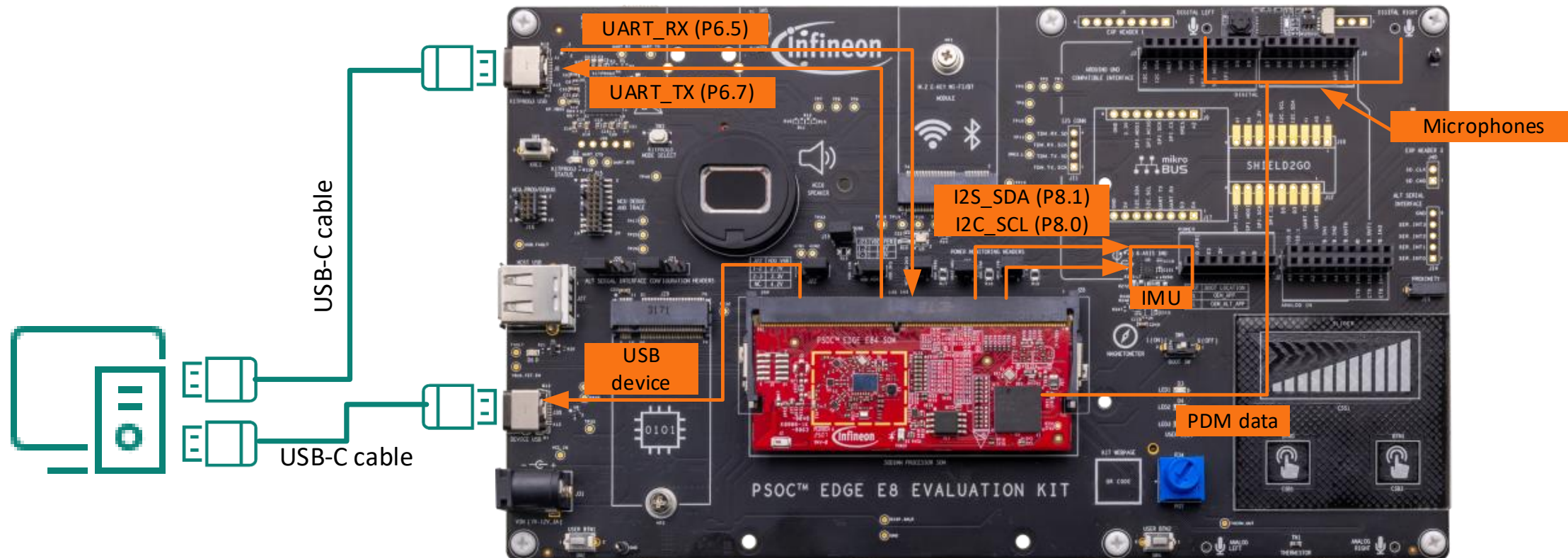


ML DEEPCRAFT™ data collection

- Utilizes [DEEPCRAFT™ streaming protocol v2](#)
- Supports collecting data from:
 - An inertial measurement unit (IMU - BMI270),
 - Magnetometer (BMM350),
 - XENSIV™ digital MEMS microphones (IM73D122V01) using PDM to PCM,
 - XENSIV™ digital barometric air pressure sensor (DPS368),
 - Digital humidity and temperature sensor (SHT40), and,
 - XENSIV™ 60 GHz radar sensor (BGT60TR13C)
- Friendly user interface using GraphUX in DEEPCRAFT™ Studio



Hardware overview



Note: the signals shown are for illustration purposes, not exact routing

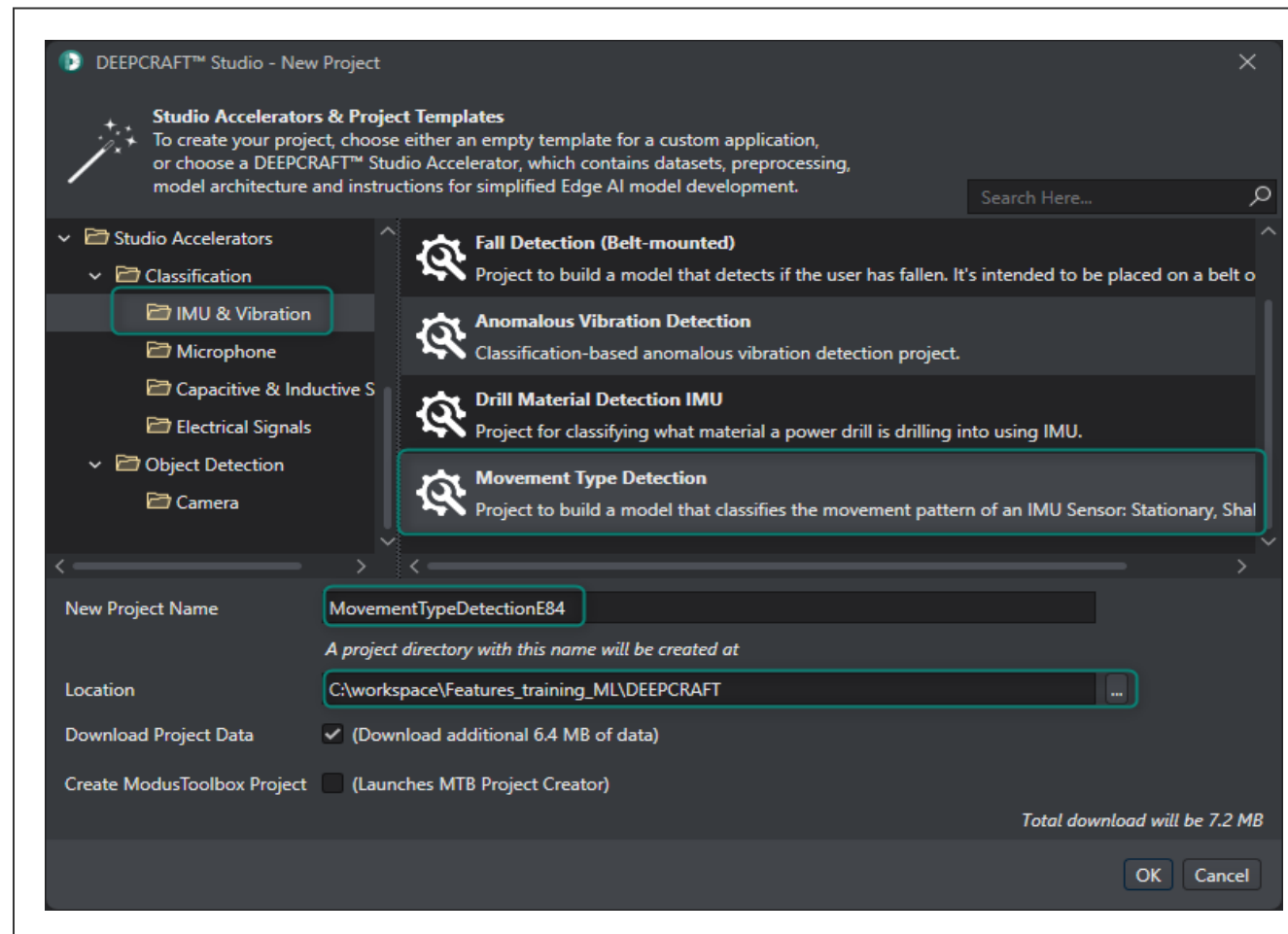
Lab 10: DEEPCRAFT™ data collection

What will you learn?

- How to collect data from an IMU, or from a digital microphone using PDM/PCM leveraging GraphUX in DEEPCRAFT™ Studio

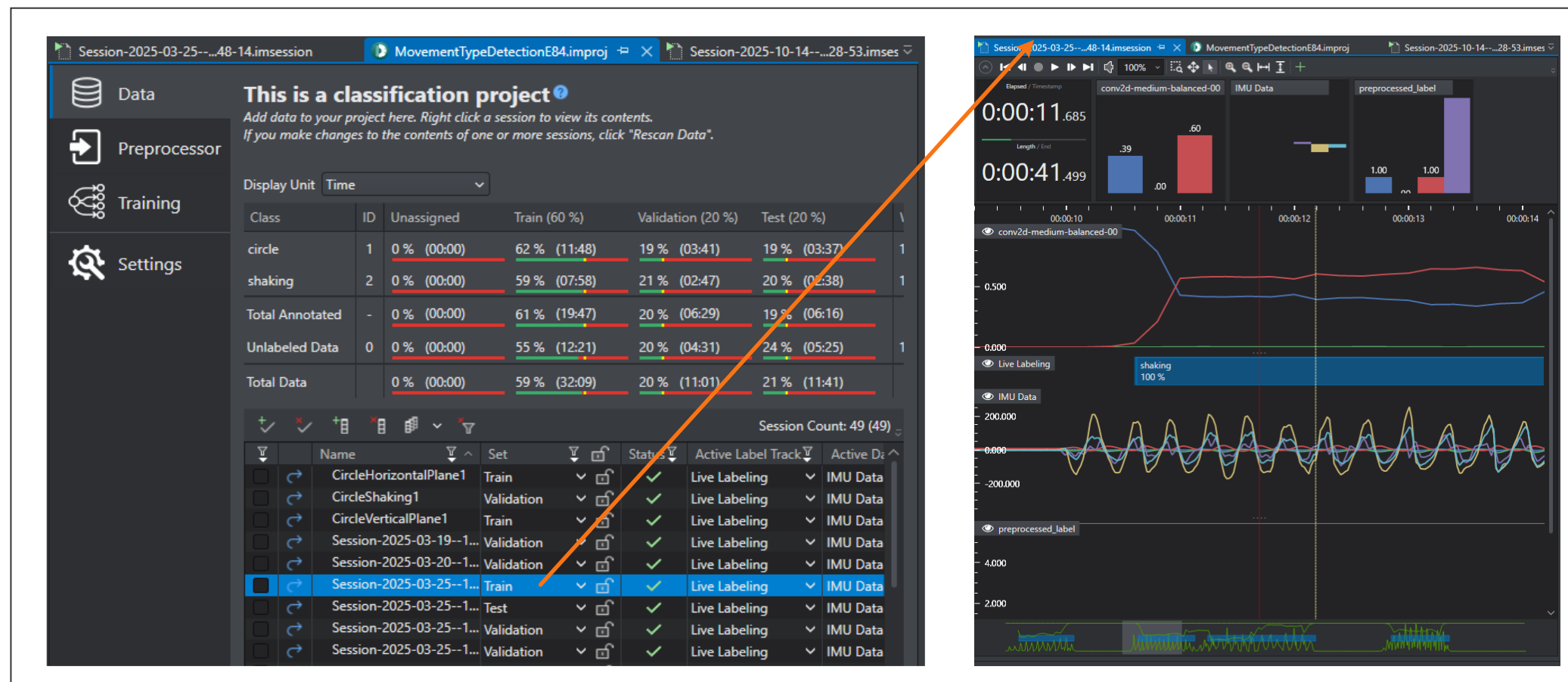
DEEPCRAFT™ Studio

- Includes reference accelerator projects, including:
 - Movement type detection (IMU)
 - Baby cry detection (PDM/PCM)
- Supports standard ML workflow
 - Data labeling
 - Setup of pre-processing pipeline
 - Model generation and training
 - Model validation
 - Model deployment

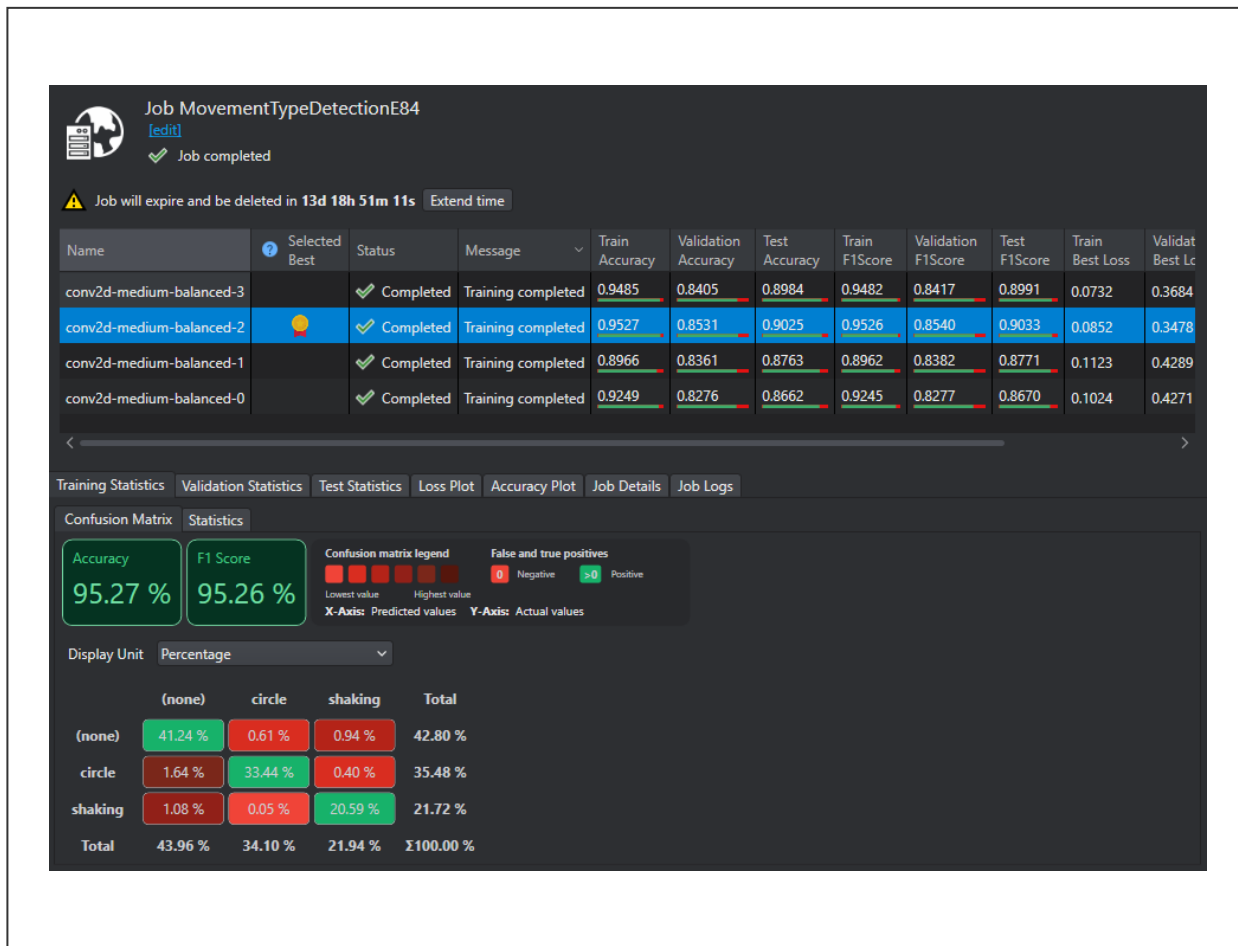


DEEPCRAFT™ Studio – data labeling

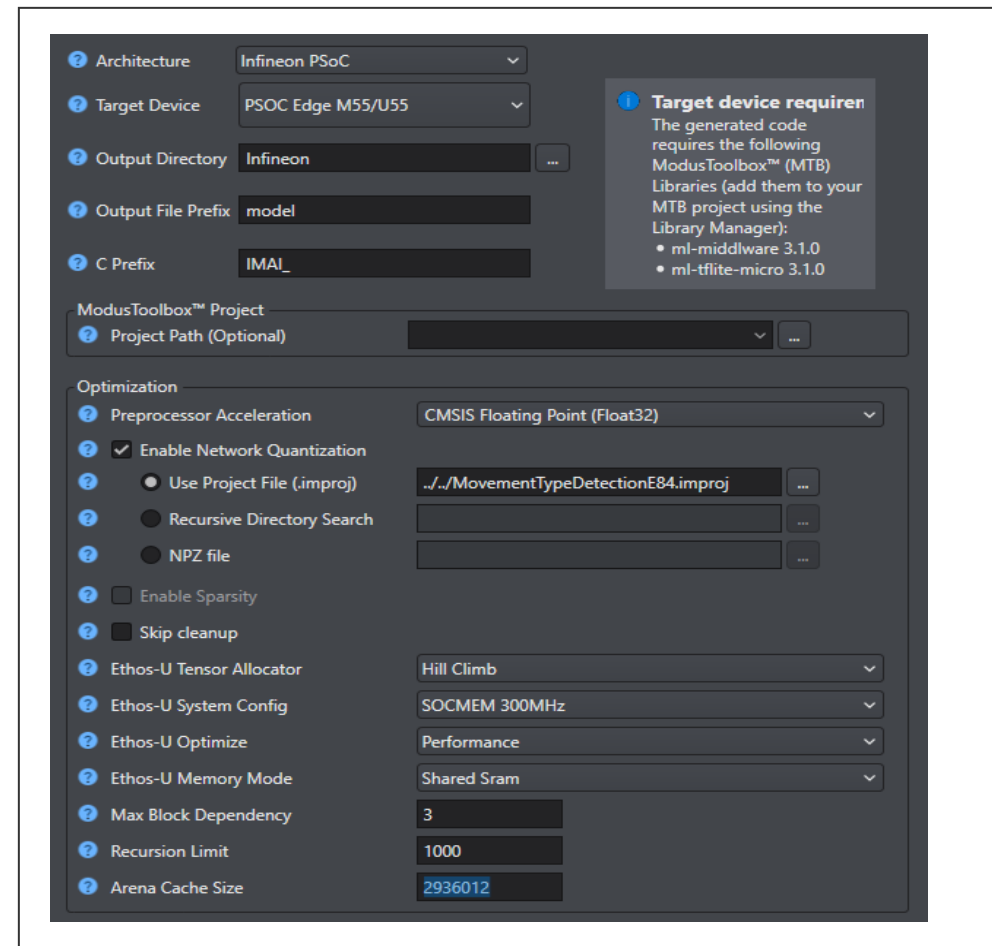
- Studio accelerators include data to accelerate development
- Intuitive process to add, label, validate and assign data to a set



DEEPCRAFT™ Studio – training and code generation



Train on the cloud comparing different models



Generate optimized code for PSOC™ Edge

Lab 10.1: DEEPCRAFT™ model generation

What will you learn?

- How to create and generate a ML model using DEEPCRAFT™ Studio

Machine learning DEEPCRAFT™ – Deploy motion/Audio

- Supports deploying an IMU **or** PDM/PCM based model
- Running a model on either the CM55/U55 **or** CM33
- Floating point or 8-bit quantization

ModusToolbox™ configurations

File	Parameter	Options	Description
common.mk	ML_DEEPCRAFT_CPU	cm55 or cm33	Switch between running the model on cm55 or cm33 CPUs. Default: cm55
<i>proj_cmxx/Makefile</i>	NN_TYPE	float or int8x8	Select between float or int8 quantization. Default: int8x8

Lab 11: DEEPCRAFT™ deployment

What will you learn?

- How to deploy a model onto a PSOC™ Edge E84 MCU device
- How to run the model on Cortex®-M33/M55

Observe results

```
VT COM41 - Tera Term VT
File Edit Setup Control Window Help
DEEPCRAFT Studio Deploy Motion Example - CM55
label: unlabelled: score: 0.292969
label: circle : score: 0.628906
label: shaking : score: 0.082031
Output: circle
█
```

Movement type detection using IMU

```
VT COM41 - Tera Term VT
File Edit Setup Control Window Help
DEEPCRAFT Studio Deploy Audio Example - CM55
label: unlabelled : score: 0.0391
label: baby_cry : score: 0.9609
Output: baby_cry
█
```

Baby crying detection using PDM/PCM

Revision history

Document revision	Date	Description of change
**	2026-01-15	Initial release.

